

UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA RAČUNARSTVO I INFORMATIKU

PageRank algoritam

ZAVRŠNI RAD

- PRVI CIKLUS STUDIJA -

Student: Zerina Aščerija
Mentor:
Red. prof. dr Željko Jurić, dipl. ing. el.

Sarajevo,
septembar 2020.

Sažetak

Potreba da se pronađe metod rangiranja stranica koji će iskoristiti informacije sadržane u hipertekstu i nužnost da se zaustavi manipulacija postojećih algoritama rangiranja, uslovila je pojavu PageRank algoritma, jezgra najpopularnijeg pretraživača, Google-a. PageRank je relativna mjera kvaliteta, značaja ili relevantnosti stranice. Matematički posmatrano, PageRank model predstavlja posebnu vrstu diskretnog stohastičkog procesa koji se naziva Markovljev lanac. Rješavanje PageRank problema svodi se na dva poznata problema: pronalaženje svojstvenih vrijednosti i rješavanje sistema linearnih jednačina. Matrica kojom se modelira problem je velika i sadrži mnogo nultih elemenata, stoga su umjesto direktnih, razvijeni iterativni metodi za rješavanje, a koji konvergiraju pod određenim uslovima i u ovisnosti od vrijednosti pojedinih parametara. Najjednostavniji i najpoznatiji je metod potencija. U situacijama kada metod potencija sporo konvergira koriste se metodi za akceleraciju kao što su Aitkenova i Kvadratna ekstrapolacija. Dodatna ubrzanja mogu se postići primjenom linearnih stacionarnih iterativnih metoda, a koji su iterativne sheme za rješavanje sistema linearnih jednačina. Varijacijama teleportacijskog vektora dobija se personalizirani rang u okviru nekog konteksta ili teme.

Ključne riječi: Stohastički proces, Svojstvene vrijednosti, Svojstveni vektori, Markovljev lanac, Perron-Frobenius, Web graf, Metod potencija

Abstract

The urge to find a page ranking method that will use the information carried by hypertext and the necessity to stop the manipulation of existing ranking algorithms, led to the development of the PageRank algorithm, the core of the most popular search engine, Google. PageRank is a measure of the quality, importance or relevance of the page. Mathematically, the PageRank model is a special type of discrete stochastic process called the Markov chain. Solving PageRank problem comes down to two well-known problems: eigenvalue problem and solving a system of linear equations. The matrix used to model the problem is large and sparse, so instead of direct ones, iterative methods for solving have been developed, which converge under certain conditions and depending on the values of certain parameters. The simplest and most well-known method is power method. When the power method converges slowly, computation is accelerated using acceleration methods such as Aitkens and Quadratic extrapolation. Additional accelerations can be achieved by applying linear stationary iterative methods, which are iterative schemes for solving systems of linear equations. Variations of the teleportation vector produce a personalized rank within a context or topic.

Keywords: Stochastic process, Eigenvalues, Eigenvectors, Markov chain, Perron-Frobenius, Web graph, Power method

Elektrotehnički fakultet, Univerzitet u Sarajevu
Odsjek za računarstvo i informatiku
Red. prof. dr Željko Jurić, dipl. ing. el.
Sarajevo, septembar 2020. godine

Postavka zadatka završnog rada I ciklusa:
PageRank algoritam

Tema:
PageRank algoritam

Kurs:
Numerički algoritmi

Student:
Zerina Aščerija

Opis:
Analizirati matematičku pozadinu PageRank algoritma što uključuje teoriju grafova, linearnu algebru i teoriju vjerovatnoće, te isti implementirati u programskom jeziku C++. Program treba sadržavati klase za modeliranje strukture grafa te pristupne metode za variranje parametara modela. Potrebno je podržati učitavanje informacija o grafu iz tekstualne datoteke, upisivanje rezultata u datoteku, te generisanje testnih skupova.

Polazna literatura:

[1] Brin, S., Lawrence Page, F., Raghavan, P., Rajagopalan, S., Stata, R., Tomkins, A., Wiener,J., "The anatomy of a large-scale hypertextual web search engine", 1998

[2] Peretti, A., Roveda, A., "On the mathematical background of google pagerank algorithm", 2014

Red. prof. dr Željko Jurić, dipl. ing. el.

Univerzitet u Sarajevu
Elektrotehnički fakultet
Odsjek za računarstvo i informatiku

Izjava o autentičnosti radova

Završni rad
I ciklusa studija

Ime i prezime: Zerina Aščerija
Naslov rada: PageRank algoritam
Vrsta rada: Završni rad Prvog ciklusa studija
Broj stranica: 48

Potvrđujem:

- da sam pročitao dokumente koji se odnose na plagijarizam, kako je to definirano Statutom Univerziteta u Sarajevu, Etičkim kodeksom Univerziteta u Sarajevu i pravilima studiranja koja se odnose na I i II ciklus studija, integrirani studijski program I i II ciklusa i III ciklus studija na Univerzitetu u Sarajevu, kao i uputama o plagijarizmu navedenim na web stranici Univerziteta u Sarajevu;
- da sam svjestan univerzitetskih disciplinskih pravila koja se tiču plagijarizma;
- da je rad koji predajem potpuno moj, samostalni rad, osim u dijelovima gdje je to naznačeno;
- da rad nije predat, u cjelini ili djelimično, za stjecanje zvanja na Univerzitetu u Sarajevu ili nekoj drugoj visokoškolskoj ustanovi;
- da sam jasno naznačio prisustvo citiranog ili parafraziranog materijala i da sam se referirao na sve izvore;
- da sam dosljedno naveo korištene i citirane izvore ili bibliografiju po nekom od preporučenih stilova citiranja, sa navođenjem potpune reference koja obuhvata potpuni bibliografski opis korištenog i citiranog izvora;
- da sam odgovarajuće naznačio svaku pomoć koju sam dobio pored pomoći mentora i akademskih tutora/ica.

Sarajevo, septembar 2020. godine

Potpis:

_____

Zerina Aščerija

Sadržaj

Popis slika **v**

Popis tabela **vi**

1 Uvod **1**
1.1 Obrazloženje teme . . . 1
1.2 Struktura rada . . . 2

2 Teoretske osnove **3**
2.1 Osnove teorije grafova . . . 3
2.2 Elementi linearne algebre . . . 4
2.3 Markovljev lanac . . . 4
2.3.1 Matrica tranzicija . . . 6
2.3.2 Konvergencija Markovljevih lanaca . . . 9
2.4 Struktura Web-a . . . 11
2.5 Pojednostavljeni PageRank algoritam . . . 13
2.5.1 Osnovne ideje . . . 13
2.5.2 Matrična reprezentacija . . . 15
2.5.3 Tumačenje ranga iz aspekta vjerovatnoće . . . 16
2.6 Generalizirani PageRank algoritam . . . 18
2.6.1 Apsorbirajući čvorovi . . . 18
2.6.2 Nepovezane komponente . . . 20

3 Implementacija **25**
3.1 Metod potencija . . . 25
3.1.1 Brzina konvergencije . . . 30
3.2 Varijacije PageRank algoritma . . . 31
3.2.1 Aitkenova ekstrapolacija . . . 31
3.2.2 Kvadratna ekstrapolacija . . . 33
3.2.3 Personalizirani PageRank algoritam . . . 36
3.2.4 BlockRank algoritam . . . 37
3.2.5 Linearni stacionarni iterativni metodi . . . 37

Prilozi **42**

Literatura **46**

Popis slika

2.1 Usmjereni graf za Primjer 1 8
2.2 Bow-tie model Web-a 12
2.3 Jednostavni graf Web-a sa 4 čvora 13
2.4 Graf sa apsorbirajućim čvorom 18
2.5 Modificirani graf sa apsorbirajućim čvorom 19
2.6 Graf sa dvije komponente povezanosti 20
2.7 Proširenje do jako povezanog grafa 21

3.1 Primjer grafa za komparaciju 29
3.2 Ovisnost broja iteracija o koeficijentu d 31

Popis tabela

3.1 Iteracije pri standardnom pristupu 29
3.2 Iteracije pri modificiranom pristupu 30
3.3 Vremenska ušteda primjenom kvadratne ekstrapolacije 36
3.4 Rastavljanje matrice za Jacobi, Gauss-Seidel i SOR metod 39
3.5 Iterativna shema Jacobi, Gauss-Seidel i SOR metoda 39

Poglavlje 1

Uvod

1.1 Obrazloženje teme

U vrijeme kada Web raspolaže ogromnom količinom informacija, od izuzetne važnosti je iz tog mnoštva efikasno izdvojiti one koje su relevantne i kvalitetne. Kako bi pratili razvoj Web-a, neophodno je da pretraživači sadržaja konstantno skaliraju, i inkorporiraju nove elemente u mehanizme pretrage. Jedan od prvih pretraživača, WWWW (World Wide Web Worm) 1994. godine imao je indeks (baza ključnih riječi odnosno meta podataka o stranici) od 110 000 stranica, te prosječno 1500 upita dnevno. Godine 1995, AltaVista, tada jedan od najviše korištenih pretraživača, imala je oko 300 000 upita dnevno, te indeks veličine 10 miliona stranica, da bi se taj broj 1997. povećao na 80 miliona. Rangiranje relevantnosti vršilo se analizom teksta na način da se broje pojavljivanja ključne riječi. Posljedica toga je da bi najbolji izbor za pretragu riječi "Oxford", na primjer, bila stranica sa najvećom učestalosti te riječi (ako bi postojala takva stranica), a ne službena stranica Oxford-a. Dakle, jasno je kako se mehanizmi pretraživanja mogu prevariti.

Osim indeksiranja, koristila se i hijerarhijska organizacija stranica (obično u imenike), pri čemu se klasifikacija vršila manuelno (Yahoo). Posljedica takvog pristupa je da su stranice uglavnom ispravno kategorisane, te je teže prevariti sistem. Međutim, posredstvo ljudskog faktora neizostavno uključuje i subjektivnost, a još veći problemi su skalabilnost, obzirom da je na takav način nemoguće pratiti rapidan rast Web-a, te troškovi i održavanje [1] [2].

U nastojanju da naprave robustan, skalabilan pretraživač koji će na automatiziran način indeksirati stranice, a pri tome biti otporan na razne strategije čiji je cilj manipulacija rangiranja, Larry Page [1] i Sergey Brin [2] razvili su PageRank algoritam, te isti implementirali u pretraživač pod imenom "Google" [3], a koji će nadmašiti konkurentne pretraživače na tržištu. Osnovna ideja PageRank algoritma jeste korištenje informacija iz hiperteksta, odnosno hiperlink/link strukture Web-a, kreiranjem Web grafa i propagaciju značaja stranice kroz grane grafa (hiperlinkove).

PageRank je vremenom evoluirao i redovno se unapređuje i mijenja (pri čemu su te varijacije tajne kako bi se spriječila zlupotreba), ali je i dalje metrika koju Google koristi zajedno sa još oko 200 kriterija za rangiranje stranica. Indeks Google-a danas sadrži stotine milijardi

[1] Lawrence "Larry" Page (East Lansing, 26. mart 1973.) je američki informatičar i poduzetnik. Zajedno sa Sergeyom Brinom je osnivač kompanije Google. Page je Generalni direktor (CEO) Google-a.

[2] Sergey Mikhaylovich Brin (August 21, 1973); je američki naučnik i poduzetnik.

[3] Google je alias za googol (broj 10^{100}), a odražava nastojanje izgradnje velikog i skalabilnog sistema

stranica, veličine je 100 000 000 GB, a dnevno odgovara na cca 5.6 milijardi upita, što je indikator superiornosti i održivosti predložene strategije, ali i motivacija da se radi na razvoju, poboljšanjima i novim strategijama, kako bi se očuvala upotrebljivost i efikasnost unatoč rastu Web-a.

Već dvije decenije od predstavljanja originalnog PageRank modela, naučna zajednica aktivno radi na proučavanju različitih pristupa, te primjeni ovog modela na druge probleme iz raznih sfera ljudskog djelovanja.

1.2 Struktura rada

Rad se sastoji od teoretskog i implementacijskog dijela, gdje u drugom dijelu zaključke i zapažanja iz prvog dijela prilagođavamo izvedbi na računaru.

U prvom, teoretskom dijelu rada, govorit ćemo o makroskopskoj strukturi Web-a i matematičkoj pozadini algoritma, te rezonu i opravdanosti uvođenja pojedinih elemenata u model, a koji omogućavaju prevazilaženje ograničenja opisane strukture. Teoretski uvod uključuje osnove teorije grafova neophodne da se razumije grafovska reprezentacija modela te ispitaju određena grafovska svojstva. Nadalje, uvod obuhvata i elemente linearne algebre i vjerovatnoće, obzirom da je srž modela sadržana u procesima i pojavama koji su predmet proučavanja navedenih nauka. Analizirat ćemo egzistenciju, jedinstvenost, osjetljivost i konvergenciju rješenja, kao i različite metode rješavanja.

U drugom dijelu rada osvrnut ćemo se na implementacijske aspekte osnovnog modela sa fokusom na smanjenju vremenske kompleksnosti sa $O(n^2)$ na $O(m)$, te modifikacije istog sa ciljem minimizacije memorijskog zauzeća i broja iteracija. Dodatno, razmotrit ćemo dvije metode ekstrapolacije za ubrzanje konvergencije i stabilizaciju rješenja kada osnovni model sporo konvergira. Na kraju, spomenut ćemo još neke varijacije algoritma, te područja primjene istih.

Poglavlje 2

Teoretske osnove

Obzirom da ćemo u nastavku koristiti grafovsku i matričnu reprezentaciju problema, te posmatrati procese iz teorije vjerovatnoće, u sljedećim potpoglavljima nalaze se neophodne definicije iz navedenih oblasti.

2.1 Osnove teorije grafova

Definicija 2.1.1 (Graf) *Graf predstavlja uređeni par $G = (X,R)$, pri čemu je X skup čvorova grafa, a skup R binarna relacija $R \subseteq \{(x_i,x_j) \mid x_i,x_j \in X\}$. Elementi (x_i,x_j) predstavljaju grane grafa, pri čemu vrijedi da $(x_i,x_j) \in R$ ako i samo ako su elementi x_i i x_j povezani granom. Ako je relacija R simetrična, graf G je neusmjeren, a ako je relacija antisimetrična, graf je usmjeren [3] [4].*

Definicija 2.1.2 (Ulazni i izlazni stepen grafa) *Ulazni stepen čvora x $d_g^-(x)$ grafa G predstavlja broj grana koje ulaze u čvor x, a izlazni stepen $d_g^+(x)$ broj grana koje izlaze iz čvora:*

$$d_G^-(x) = \#\{y \,|\, (y,x) \in R\} \tag{2.1}$$

$$d_G^+(x) = \#\{y \,|\, (x,y) \in R\}[4]. \tag{2.2}$$

Definicija 2.1.3 (Put u grafu) *Put u grafu $G = (X,R)$ je svaka sekvenca grana $g_1,g_2,...,g_k$, (g_i predstavlja element skupa R tj. $(x_i,x_j) \in R$), pri čemu svaka grana g_i, $i = 2,3,...,k$, počinje u onom čvoru u kojem završava grana g_{i-1}. Početni čvor grane g_1, i krajnji čvor grane g_k nazivaju se **početak** i **kraj** puta. Put koji ne sadrži više puta istu granu naziva se **staza**. Put je **zatvoren** ukoliko mu se početak i kraj poklapaju, inače je otvoren. Zatvorena staza naziva se **ciklus** [4].*

Definicija 2.1.4 (Podgraf grafa) *Graf $G' = (X',R')$ je podgraf drugog grafa $G = (X,R)$ akko:*
$X' \subseteq X$, *i*
$R' \subseteq R \wedge ((x_1,x_2) \in R' \implies x_1,x_2 \in X')$ *[3].*

Definicija 2.1.5 (Povezanost dva čvora) *Ukoliko postoji put kojem su čvorovi x_i i x_j početak i kraj, kažemo da su čvorovi x_i i x_j **povezani**, u suprotnom su **nepovezani** [4].*

Definicija 2.1.6 (Komponenta povezanosti) *Svaki podgraf koji je obrazovan od svih čvorova koji su povezani s nekim izabranim čvorom čini jednu **komponentu povezanosti** grafa [4].*

Definicija 2.1.7 (Jaka povezanost) *Dva čvora x_i i x_j su **jako povezana** ako postoji put iz čvora x_i u čvor x_j, te put iz čvora x_j u čvor x_i. **Komponente jake povezanosti** grafa čine podgrafovi koji su obrazovani od svih čvorova koji su jako povezani s nekim izabranim čvorom grafa [4].*

2.2 Elementi linearne algebre

Definicija 2.2.1 (Pozitivnost i nenegativnost matrice) *Matrica* $\mathbf{A}$ *je pozitivna ako su svi njeni elementi pozitivni, odnosno* $a_{ij} > 0, \quad \forall i, j$. *Matrica* $\mathbf{A}$ *je nenegativna ako su svi njeni elementi nenegativni, odnosno* $a_{ij} \geq 0, \quad \forall i, j$.

Definicija 2.2.2 (Svojstvene vrijednosti) *Neka je zadana* $n \times n$ *realna ili kompleksna matrica* $\mathbf{A}$. *Realni ili kompleksni broj* λ *i vektor* $\mathbf{v} \neq \mathbf{0}$ *koji su netrivijalna rješenja jednačine:*

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

nazivamo svojstvena vrijednost i svojstveni vektor matrice $\mathbf{A}$. *Kažemo da je svojstveni vektor (desni)* $\mathbf{v}$ *pridružen svojstvenoj vrijednosti* λ. *Analogno, vektor* ω *je lijevi svojstveni vektor pridružen svojstvenoj vrijednosti* λ*:*

$$\omega^T\mathbf{A} = \lambda\omega^T [5].$$

Za određivanje svojstvenih vrijednosti i vektora potrebno je riješiti jednačinu: $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{I}\mathbf{v} \implies (\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$$

gdje je $\mathbf{I}$ jedinična matrica $n \times n$. Trivijalno rješenje je $\mathbf{v} = \mathbf{0}$, i vrijedi $\forall\lambda$, te stoga nije od interesa. Ako postoji i netrivijalno rješenje, znači da trivijalno rješenje nije jedinstveno, odnosno da je matrica $\mathbf{A} - \lambda\mathbf{I}$ singularna. Matrica je singularna akko je njena determinanta jednaka 0, što znači da vrijedi:

$$det(\mathbf{A} - \lambda\mathbf{I}) = 0$$

Svojstvene vrijednosti matrice $\mathbf{A}$ su korijeni karakterističnog (svojstvenog) polinoma $P_A(\lambda) = det(\mathbf{A} - \lambda\mathbf{I})$. Višestrukost korijena λ_a karakterističnog polinoma naziva se algebarska višestrukost $m_a(\lambda)$.

Definicija 2.2.3 (Spektar matrice) *Skup svih svojstvenih vrijednosti* λ *(za koje je* $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ *za neki* $\mathbf{v} \neq 0$*) zove se spektar matrice* $\mathbf{A}$.

2.3 Markovljev lanac

Markovljev lanac je matematički sistem koji vrši tranzicije iz stanja u stanje prema, unaprijed određenim, pravilima vjerovatnoće. Osnovna karakteristika Markovljevog lanca je da nije bitno kako je sistem dospio u neko stanje: vjerovatnoće budućih stanja su uvijek fiksne, odnosno prelazak u neko drugo stanje je uvjetovan isključivo trenutnim stanjem. Skup mogućih stanja može biti bilo šta: vremenski uslovi, performanse na berzi, popularnost trgovačkih lanaca itd. Iako je moguće razmatrati skupove stanja bilo koje kardinalnosti, većina aplikacija je zasnovana na konačnim skupovima. Markovljevi lanci su primjenjivi kao statistički modeli realnih procesa u raznim sferama.

Markovljev lanac je *stohastički proces* (vidjeti 2.3.5), ali se razlikuje od općeg stohastičkog procesa u smislu da Markovljev lanac mora biti "bez memorije", odnosno sistem pamti samo trenutno stanje, te s tim u vezi, buduće akcije ovise isključivo o trenutnom stanju. U nastavku ćemo definisati neophodne elemente iz teorije vjerovatnoće te uvesti formalnu definiciju Markovljevog lanca.

Definicija 2.3.1 (Događaj) *Bilo kakva radnja koja može imati jedan ili više, ali najviše prebrojivo mnogo ishoda, predstavlja* ***slučajni eksperiment****. Neprazni skupovi ishoda nazivaju se* ***događaji****. Događaji koji uključuju samo jedan ishod nazivaju se* ***elementarni događaji*** *[6].*

Definicija 2.3.2 (Vjerovatnoća) *Neka je F familija događaja na prostoru elementarnih događaja Ω. Preslikavanje P koje svakom događaju A iz F pridružuje realan broj $P(A)$ tako da vrijedi:*

(i) $0 \leq P(A) \leq 1$

(ii) $P(\Omega) = 1$

(iii) Za međusobno disjunktne događaje $A_1, A_2, ..., A_n$ iz F vrijedi:

$$P(A_1 \cup A_2 ... \cup A_n) = P(A_1) + P(A_2) + ... + P(A_n) \tag{2.3}$$

zovemo ***vjerovatnoća*** *na (Ω, F). Broj P(A) zovemo vjerovatnoća događaja A. Uređenu trojku (Ω, F, P) zovemo vjerovatnosni prostor [7].*

Definicija 2.3.3 (Uvjetna vjerovatnoća) *Neka je (Ω, F, P) bilo koji vjerovatnosni prostor, te neka su A, B dva događaja. Pretpostavimo da znamo da se događaj B dogodio ($P(B) > 0$).* ***Uvjetna vjerovatnoća*** *događaja A, uz uvjet da se dogodio B, definira se kao broj*

$$P(A|B) := \frac{P(A \cap B)}{P(B)} \tag{2.4}$$

Definicija 2.3.4 (Slučajna varijabla) *Slučajna varijabla X je funkcija koja svakom elementarnom događaju pridružuje realan broj:*

$$X : \Omega \to \mathbb{R}$$

Sa ImX označavamo sliku varijable X, odnosno skup vrijednosti koje slučajna varijabla poprima. Slučajna varijabla je ***diskretna*** *ako je ImX prebrojiv skup [7].*

Definicija 2.3.5 (Stohastički proces) *Neka je (Ω, F, P) vjerovatnosni prostor, i neka je $I \subset \mathbb{R}$. Nadalje pretpostavimo da $\forall \alpha \in I$ postoji slučajna varijabla $X_\alpha : \Omega \to \mathbb{R}$ definirana na (Ω, F, P). Funkcija $X : I \times \Omega \to \mathbb{R}$ definisana sa $X(\alpha, \omega) = X_\alpha(\omega)$ naziva se* ***stohastički proces*** *indeksiran skupom I.*

Neformalno, stohastički proces je familija slučajnih varijabli $\{X(t) : t \in I\}$, pri čemu t uglavom označava vrijeme. Stohastički proces je **diskretan** ako je skup I konačan ili prebrojiv.

Definicija 2.3.6 (Markovljev lanac) *Sekvencu $X_0, X_1, X_2, ...$ diskretnih slučajnih varijabli nazivamo* ***Markovljevim lancem*** [1] *ako vrijednost X_{t+1} ovisi isključivo od prethodnog stanja X_t:*

$$P(X_{t+1} = s | X_t = s_t, X_{t-1} = s_{t-1}, ..., X_0 = s_0) = P(X_{t+1} = s | X_t = s_t) \tag{2.5}$$

za svako $t = 1, 2, 3, ...$ i za sva stanja $s_0, s_1, ..., s_t, s$. Sekvenca $s_0, s_1, ..., s_t, s$ predstavlja prebrojiv skup stanja $S = \{s, s_1, s_2, ..., s_n\}$ koji se naziva ***prostor stanja lanca****. Ovo svojstvo naziva se* ***Markovljevo svojstvo****.*

[1] Andrej Andrejevič Markov (14. juni 1856. - 20. juli 1922.) je bio ruski matematičar najpoznatiji po istraživanjima u teoriji stohastičkih procesa.

Dakle *Markovljev lanac* predstavlja *diskretni stohastički proces*. Važno je naglasiti da svojstvo neovisnosti X_{t+1} o X_0, $X_1, ..., X_{t-1}$ implicira da se sva ovisnost o prijašnjim stanjima nalazi u X_t.

Kako je X_t slučajna varijabla, ona ima određenu vjerovatnosnu distribuciju (engl. probability distribution). Distribuciju slučajne varijable X_t predstavljaju vjerovatnoće da će slučajna varijabla poprimiti vrijednost nekog od stanja, a možemo je posmatrati kao $n \times 1$ vektor koji ćemo označavati sa π. Na primjer, posmatrajmo distribuciju za varijablu X_0:

$$\pi = \begin{pmatrix} \pi_1 \\ \pi_2 \\ \vdots \\ \pi_n \end{pmatrix} = \begin{pmatrix} P(X_0 = s_1) \\ P(X_0 = s_2) \\ \vdots \\ P(X_0 = s_n) \end{pmatrix}$$

Koristit ćemo notaciju $X \sim \pi$ da označimo da je vektor π vjerovatnosna distribucija slučajne varijable X.

Definicija 2.3.7 (Homogeni Markovljev lanac) *Markovljev lanac je **homogen (vremenski invarijantan, stacionaran)** ako $\forall n, l$ vrijedi:*

$$P(X_n = a | X_{n-1} = b) = P(X_{n+1} = a | X_{n+l-1} = b), \quad a, b \in \Omega \tag{2.6}$$

Dakle tranzicije ne ovise o vremenskom trenutku nastupanja (parametar l označava vremenski pomak), te se homogeni Markov lanac može specificirati početnom distribucijom X_0 i vjerovatnoćama tranzicija jer ostaju konstantne tokom vremena. U nastavku ćemo *homogeni Markovljev lanac* nazivati jednostavno *Markovljev lanac*.

Neka je $\pi = [\pi_1, \pi_2, ..., \pi_n]^T$ distribucija slučajne varijable X_0. Neka nas zanima vjerovatnoća da u sljedećem trenutku sistem dospije u stanje $s_i, \quad i \leq n$ (efektivno to znači da će slučajna varijabla X_1 poprimiti vrijednost s_i). Ta vjerovatnoća iznosi:

$$P(X_1 = s_i) = P(X_0 = s_1)P(X_1 = s_i | X_0 = s_1) + P(X_0 = s_2)P(X_1 = s_i | X_0 = s_2) + ... \\ + P(X_0 = s_n)P(X_1 = s_i | X_0 = s_n)$$

odnosno:

$$P(X_1 = s_i) = \pi_1 P(X_1 = s_i | X_0 = s_1) + \pi_2 P(X_1 = s_i | X_0 = s_2) + ... + \pi_n P(X_1 = s_i | X_0 = s_n)$$

$$P(X_1 = s_i) = \sum_{j=1}^{n} P(X_0 = s_j)P(X_1 = s_i | X_0 = s_j), \quad i = 1, 2, ...n \tag{2.7}$$

Dakle radi se o totalnoj vjerovatnoći, a u nastavku ćemo razmotriti, kompaktniji, matrični zapis.

2.3.1 Matrica tranzicija

Markovljeve lance često predstavljamo kao matrice čiji su elementi vjerovatnoće tranzicija. Tu matricu nazivamo matrica tranzicija (engl. transition matrix), i označavamo je sa **P**. Neka je skup stanja $S = \{s_1, s_2, ..., s_n\}$, koje ćemo skraćeno označavati $\{1, 2, ..., n\}$. U matrici **P**:

- kolone predstavljaju trenutno stanje (engl. FROM), odnosno stanje sa koga se vrši tranzicija (X_t)

- redovi predstavljaju buduće stanje (engl. TO), odnosno stanje u koje se prelazi (X_{t+1})
- element na poziciji (i, j) je uslovna vjerovatnoća da je $X_{t+1} = i$, ako je $X_t = j$, odnosno vjerovatnoća tranzicije iz stanja j u stanje i:

$$p_{ij} = P(X_{t+1} = i | X_t = j) \tag{2.8}$$

Matrica **P** ima sljedeće osobine:

(i) Sva stanja iz prostora stanja S moraju biti sadržana u matrici **P**

(ii) Matrica **P** mora biti kvadratna $(n \times n)$, obzirom da X_{t+1} i X_t poprimaju vrijednosti iz istog skupa stanja S (kardinalnosti n)

(iii) Suma elemenata po kolonama mora biti jednaka 1:

$$\sum_{i=1}^{n} p_{ij} = \sum_{i=1}^{n} P(X_{t+1} = i | X_t = j) = 1, \quad j = 1, 2, ...n \tag{2.9}$$

Odnosno, iz stanja j u sva druga se prelazi sa određenom vjerovatnoćom, a jasno je da zbir svih vjerovatnoća mora biti 1. Matrice koje zadovoljavaju ovaj uslov, nazivaju se *stohastičke po kolonama*.

(iv) Suma elemenata po redovima ne mora biti 1.

Nakon što smo pokazali kako se konstruiše matrica **P**, jasno je da formulu 2.7, možemo zapisati u matričnom obliku kao:

$$\begin{aligned} P(X_1 = i) &= \textstyle\sum_{j=1}^{n} P(X_0 = j) P(X_1 = i | X_0 = j) = \\ &= \textstyle\sum_{j=1}^{n} P(X_1 = i | X_0 = j) P(X_0 = j) = \\ &= \textstyle\sum_{j=1}^{n} p_{ij} \pi_j = (\mathbf{P}\pi)_i, \quad i = 1, 2, ...n \end{aligned}$$

Notacija $(\mathbf{P}\pi)_i$ označava množenje $i-tog$ reda matrice **P** vektor kolonom π. Obzirom da $P(X_1 = i)$ predstavlja $i-ti$ element vektora vjerovatnosne distribucije slučajne varijable X_1, možemo zaključiti da vrijedi:

$$X_1 \sim \mathbf{P}\pi \tag{2.10}$$

Ako bismo isti postupak ponovili za slučajnu varijablu X_2, dobili bismo da vrijedi: $X_2 \sim \mathbf{P}^2\pi$, čime dolazimo do teorema:

Teorem 2.3.1 *Neka je $\{X_0, X_1, X_2, ...\}$ Markovljev lanac sa $n \times n$ matricom tranzicija* **P**. *Ako je vjerovatnosna distribucija slučajne varijable X_0 data vektorom π dimenzija $n \times 1$, onda je vjerovatnosna distribucija slučajne varijable X_t data vektorom $\mathbf{P}^t\pi$ dimenzija $n \times 1$* [8]. *Odnosno:*

$$X_0 \sim \pi \implies X_t \sim \mathbf{P}^t\pi. \tag{2.11}$$

Osim matrica, česta je reprezentacija usmjerenim grafom $G = (X, R)$ gdje čvorovi označavaju elemente prostora stanja, a grane sadrže informaciju o vjerovatnoći tranzicije iz početnog čvora u krajnji čvor grane, pri čemu je suma vjerovatnoća svih izlaznih grana čvora jednaka 1:

$$X = S$$

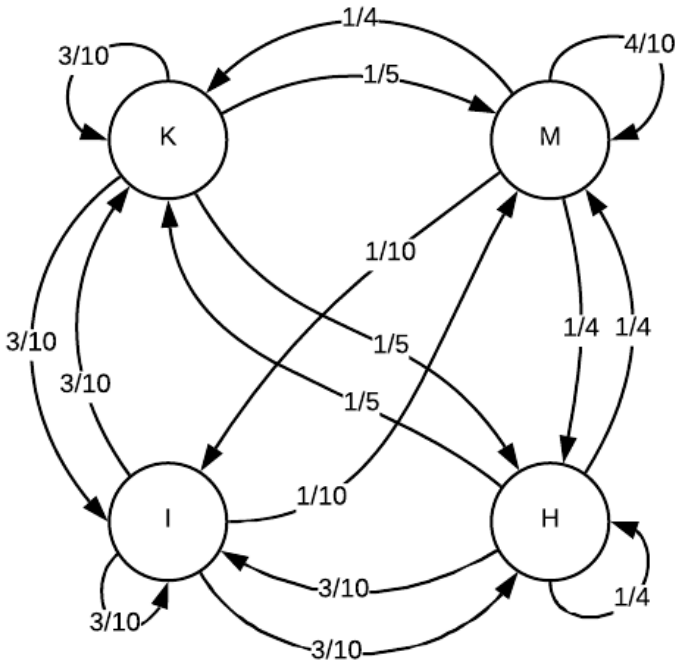
$$R = \{(i, j) | P_{i,j} > 0\}$$

Na primjeru ispod demonstrirat ćemo izloženo.

Primjer 1: U nekom malom gradu otvoreni su Kineski, Meksički i Italijanski restoran Svaki građanin jede u nekom od ta 3 restorana ili kod kuće. Pretpostavimo da 20% onih koji jedu u Kineskom restoranu sljedeći put jedu u Meksičkom restoranu, 20% kod kuće, 30% u Italijanskom, a ostali opet posjećuju Kineski restoran. Od onih koji jedu u Meksičkom restoranu, 10% sljedeći put odlazi u Italijanski, 25% u Kineski restoran, 25% jede kod kuće, a ostatak opet u Meksičkom restoranu. Od onih koji jedu u Italijanskom restoranu, 30% sljedeći put jede kući, 30% u Kineskom restoranu, 10% u Meksičkom, a ostatak se vraća u Italijanski restoran. Od onih koji jedu kući, 20% sljedeći put jede u Kineskom restoranu, 25% u Meksičkom, 30% u Italijanskom, a ostali opet jedu kući.

Dakle, skup stanja predstavlja $S = \{K, M, I, H\}$, a koja respektivno označavaju Kineski restoran, Meksički restoran, Italijanski restoran i kuću.

Usmjereni graf koji odgovara ovom problemu prikazan je na slici 2.1:



Slika 2.1: Usmjereni graf za Primjer 1

Matrica tranzicija **P**:

$$\begin{array}{c} \begin{matrix} K & M & I & H \end{matrix} \\ \begin{pmatrix} \frac{3}{10} & \frac{1}{4} & \frac{3}{10} & \frac{1}{5} \\ \frac{1}{5} & \frac{4}{10} & \frac{1}{10} & \frac{1}{4} \\ \frac{3}{10} & \frac{1}{10} & \frac{3}{10} & \frac{3}{10} \\ \frac{1}{5} & \frac{1}{4} & \frac{3}{10} & \frac{1}{4} \end{pmatrix} \begin{matrix} K \\ M \\ I \\ H \end{matrix} \end{array}$$

Pretpostavimo, da su se restorani tek otvorili, te da svi jedu kući, pa je početna distribucija:

$$\pi = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

Neka nas zanima posjećenost svakog restorana u nekom narednom trenutku. Taj trenutak može biti sljedeća sedmica, mjesec, godina, a sve u ovisnosti od toga kakav je skup I (2.3.5) kojim indeksiramo slučajne varijable. U skladu sa 2.11, distribucija varijable X_1 iznosi:

$$X_1 \sim \mathbf{P}\pi = \begin{pmatrix} \frac{3}{10} & \frac{1}{4} & \frac{3}{10} & \frac{1}{5} \\ \frac{1}{5} & \frac{4}{10} & \frac{1}{10} & \frac{1}{4} \\ \frac{3}{10} & \frac{1}{10} & \frac{3}{10} & \frac{3}{10} \\ \frac{1}{5} & \frac{1}{4} & \frac{3}{10} & \frac{1}{4} \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{5} \\ \frac{1}{4} \\ \frac{3}{10} \\ \frac{1}{4} \end{pmatrix}$$

Nadalje, ako nas zanima distribucija u drugoj, šestoj, desetoj sedmici:

$$X_2 \sim \mathbf{P}^2\pi = \begin{pmatrix} \frac{27}{100} & \frac{51}{200} & \frac{53}{200} & \frac{21}{80} \\ \frac{11}{50} & \frac{113}{400} & \frac{41}{200} & \frac{93}{400} \\ \frac{13}{50} & \frac{11}{50} & \frac{7}{25} & \frac{1}{4} \\ \frac{1}{4} & \frac{97}{400} & \frac{1}{4} & \frac{51}{200} \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{21}{80} \\ \frac{93}{400} \\ \frac{1}{4} \\ \frac{51}{200} \end{pmatrix}$$

Analogno, za X_6, X_{10}, X_{30} imamo sljedeće rezultate:

$$X_6 \sim \begin{pmatrix} 0.2633 \\ 0.2339 \\ 0.2532 \\ 0.2495 \end{pmatrix}, \quad X_{10} \sim \begin{pmatrix} 0.2633 \\ 0.2339 \\ 0.2532 \\ 0.2495 \end{pmatrix}, \quad X_{30} \sim \begin{pmatrix} 0.2633 \\ 0.2339 \\ 0.2532 \\ 0.2495 \end{pmatrix}$$

Možemo uočiti da se vremenom vjerovatnosne distribucije ustaljuju oko vektora $[0.2633, 0.2339, 0.2532, 0.2495]^T$, međutim, to nije odlika svih Markovljevih lanaca, te ćemo u nastavku ispitati uslove pod kojima distribucije vjerovatnoće konvergiraju ka određenim vrijednostima.

2.3.2 Konvergencija Markovljevih lanaca

Konvergencija Markovljevih lanaca ovisi o nekoliko svojstava za čije dokazivanje postoji više načina [5], ali ovdje ćemo se fokusirati na grafovski smisao tih svojstava, obzirom da ista vrijede i za matricu.

Osobine koje ćemo razmatrati su:

- periodičnost stanja - stanje ima period k ako je broj koraka od napuštanja do povratka u to stanje uvijek djeljiv brojem k. Ako je $k = 1$, stanje je aperiodično, u suprotnom je periodično. Ako su sva stanja aperiodična, Markovljev lanac je aperiodičan.
- ireducibilnost - Markovljev lanac je ireducibilan akko je odgovarajući graf jako povezan.
- prolaznost stanja - stanja mogu biti ponavljajuća (trajna) ili prolazna (tranzijentna) u ovisnosti da li je uvijek moguć povratak jednom kad se stanje napusti. Stanje za koje je povratak uvijek moguć je ponavljajuće, u suprotnom je prolazno. Stanje je pozitivno ponavljajuće ako je povratak moguć u konačno mnogo koraka.

Definicija 2.3.8 (Stacionarna vjerovatnosna distribucija) *Vektor kolona π se naziva **stacionarnom vjerovatnosnom distribucijom** nad skupom stanja S, ako je π vjerovatnosna distribucija i vrijedi:*

$$\mathbf{P}\pi = \pi. \tag{2.12}$$

Jasno je da postoji sličnost između formule 2.2.2 i formule 2.12. Zaista, za $\lambda = 1$, dobijamo ekvivalentne jednačine, što znači da je *stacionarna vjerovatnosna distribucija* zapravo *svojstveni vektor* matrice $\mathbf{P}$ pridružen *svojstvenoj vrijednosti* $\lambda = 1$.

Međutim, potrebno je pokazati da matrica $\mathbf{P}$ zaista ima jedinstvenu svojstvenu vrijednost $\lambda = 1$ (neophodno je da bude jedinstvena kako bi sistem konvergirao ka jednoj vrijednosti) i da svojstveni vektor pridružen toj svojstvenoj vrijednosti može predstavljati vjerovatnosnu distribuciju (pozitivan i jedinstven). Uslovi koje matrica $\mathbf{P}$ mora zadovoljavati i posljedice opisani su teoremima u nastavku.

Teorem 2.3.2 (Perron-Frobenius teorem za nenegativne ireducibilne matrice) *Neka je $\mathbf{A}$ nenegativna ireducibilna matrica. Tada vrijedi:*

(i) Postoji realna i pozitivna svojstvena vrijednost r (poznata kao Perron-Frobenius svojstvena vrijednost ili dominantna svojstvena vrijednost) takva da za svaku drugu svojstvenu vrijednost vrijedi $|\lambda| < r$.

(ii) Algebarska višestrukost svojstvene vrijednosti r je jedan , $m_r(\lambda) = 1$.

(iii) Postoji pozitivan svojstveni vektor $\mathbf{v}$ pridružen svojstvenoj vrijednosti r.

(iv) Ne postoje drugi pozitivni svojstveni vektori matrice $\mathbf{A}$, osim onih kolinearnih vektoru $\mathbf{v}$. Analogno ne postoji ni drugi lijevi pozitivni svojstveni vektor osim ω[5].

Perron-Frobenius teorem nam za ireducibilnu matricu garantuje postojanje jedinstvene i pozitivne svojstvene vrijednosti za koju postoji pozitivan svojstveni vektor pri čemu je to jedini pozitivan vektor (odnosno za bilo koju drugu svojstvenu vrijednost ne postoji vektor čiji su svi elementi pozitivni). Obzirom da je u našem slučaju taj vektor vjerovatnosna distribucija, jasno je da mora biti pozitivan.

Definicija 2.3.9 (Regularnost matrice) *Matrica $\mathbf{A}$ je regularna ako postoji potencija matrice t tako da je matrica A^t pozitivna. Vrijedi sljedeće:*

(i) Ako matrica nije ireducibilna, onda nije ni regularna

(ii) Ako je matrica ireducibilna, i makar jedan dijagonalni element je pozitivan, onda je regularna

Teorem 2.3.3 *Neka je* $\mathbf{A}$ *regularna stohastička matrica.*

(i) Matrica $\mathbf{A}$ *ima jedinstvenu vrijednost* $\lambda = 1$ *višestrukosti 1. Postoji pozitivan vektor* $\mathbf{v}$ *pridruženoj svojstvenoj vrijednosti* $\lambda = 1$ *takav da je suma svih elemenata vektora 1.*

(ii) Za sve druge svojstvene vrijednosti λ_j *vrijedi* $|\lambda| < 1$. *Ako je vektor* $\mathbf{v}_j$ *pridružen svojstvenoj vrijednosti* λ_j, *tada je suma njegovih elemenata jednaka 0.*

Dakle, za regularnu stohastičku matricu imamo garanciju da postoji svojstvena vrijednost $\lambda = 1$ kojoj je pridružen vektor koji predstavlja vjerovatnosnu distribuciju. Međutim, jasno je da su ovi uslovi poprilično jaki, ali imaju svoje tumačenje.

Naime, Markovljev lanac je *aperiodičan* akko postoji prirodan broj t, takav da su svi elementi odgovarajuće matrice tranzicija $\mathbf{P}^t$ striktno pozitivni, što znači je uslov na aperiodičnost Markovljevog lanca ekvivalentan uslovu na regularnost matrice $\mathbf{P}$.

Uslovom na ireducibilnost pokazali smo postojanje i jedinstvenost pozitivnog svojstvenog vektora. Međutim, to nam ništa ne govori o konvergenciji ka tom vektoru tokom vremena. U prisustvu periodičnih i trajnih stanja, pri čemu su kod ireducibilnih grafova sva stanja trajna, zapravo nikad ne gubimo informacije o početnom stanju, u našem slučaju vjerovatnosnoj distribuciji slučajne varijable X_0. To bi značilo da bismo morali krenuti od neke distribucije koja će vremenom konvergirati ka traženom vektoru. Aperiodičnost nam garantuje da će vjerovatnosna distribucija konvergirati ka stacionarnoj distribuciji bez obzira na početnu distribuciju varijable X_0.

Konačno, dolazimo do zaključka: Svaki ireducibilan aperiodičan Markovljev lanac sa konačno mnogo stanja ima jedinstvenu stacionarnu distribuciju prema kojoj konvergira bez obzira na početnu distribuciju. Dokaz se nalazi u [9].

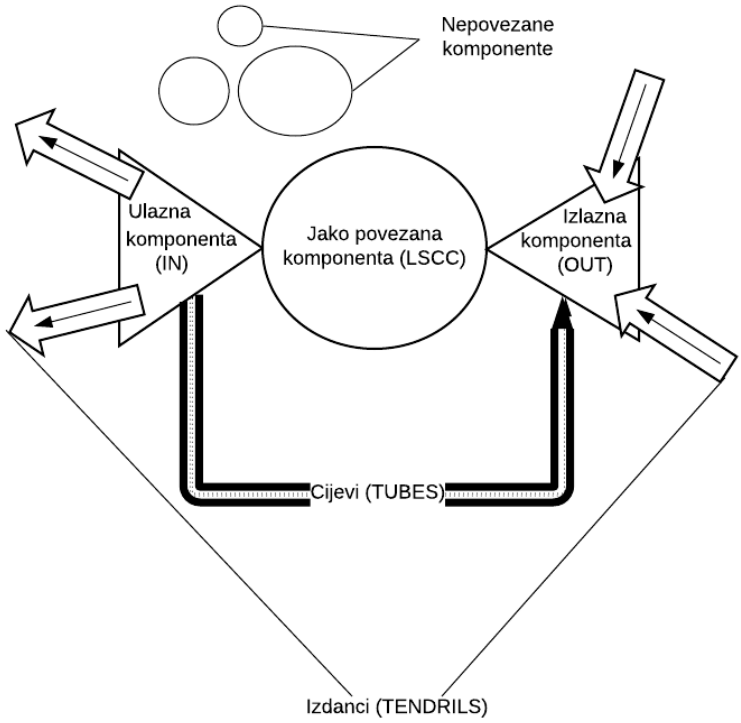
2.4 Struktura Web-a

World Wide Web (WWW), poznatiji kao Web, predstavlja složeni informacioni sistem, sačinjen od web stranica, a koje obično sadrže hiperlinkove (linkove, reference) na druge stranice, koje korisnik "slijedi" da pristupi sadržaju na koji hiperlink upućuje. Bez obzira na kompleknost i veličinu, Web možemo predstaviti kao usmjereni graf čiji su čvorovi web stranice, a grane hiperlinkovi između stranica. A. Broder i drugi [10] predlažu bow-tie model sačinjen od 6 komponenti:

- Komponenta jake povezanosti grafa (Large strong connnected component - LSCC) - centralna jezgra koju sačinjavaju stranice (čvorovi) koje su jako povezane hiperlinkovima.
- Ulazna komponenta (IN) - sačinjavaju je stranice sa kojih je dostupna centralna jezgra (putem hiperlinkova), ali one nisu dostupne iz centralne jezgre. Obično su to nove, neotkrivene stranice.
- Izlazna komponenta (OUT) - sačinjavaju je stranice koje su dostupne iz centralne jezgre, ali one ne upućuju nazad na jezgru. Uglavnom su to korporativne stranice sa internim linkovima isključivo.
- Izdanci (TENDRILS) - sačinjavaju je stranice koje nisu dostupne iz centralne jezgre, niti upućuju na jezgru. Stranice ulazne komponente mogu upućivati na njih, ili one mogu sadržavati linkove na izlaznu komponentu.

- Cijevi (TUBES) - sačinjavaju ih stranice koje pripadaju ulaznoj komponenti, a sadrže reference na stranice izlazne komponente.
- Nepovezane komponente - sačinjavaju ih stranice koje su izolovane od ostalih komponenti.

  Na slici 2.2 prikazana je makroskopska struktura Web-a u skladu sa bow-tie modelom.



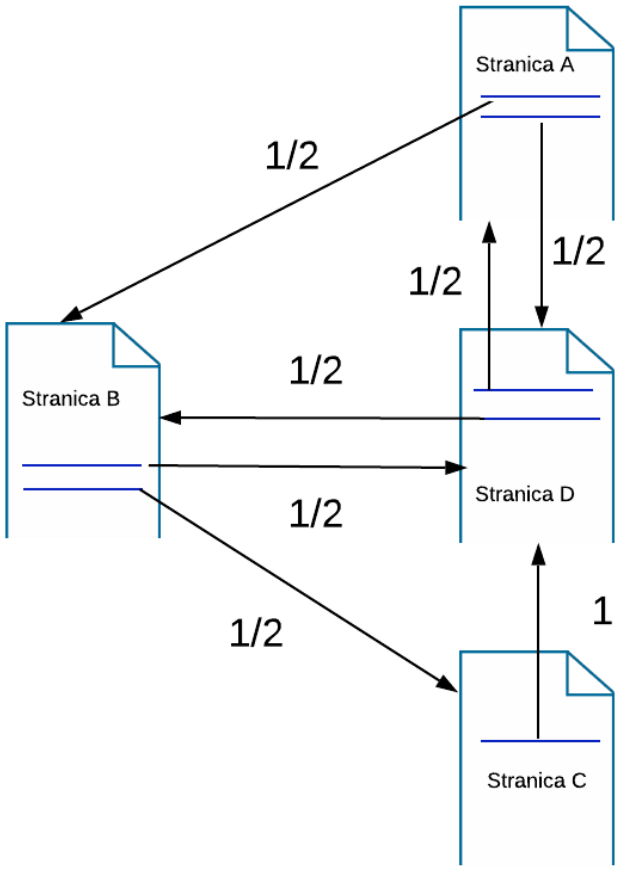
Slika 2.2: Bow-tie model Web-a

U nastavku ćemo, prethodno razmatranu strukturu, apstrahovati i koristiti grafove sačinjene od malog broja čvorova i grana u cilju što jednostavnije demonstracije koncepata, ali i inkrementalno vršiti korekcije nad predloženim modelima, kako bismo osigurali upotrebljivost istih uprkos složenosti Web grafa.

2.5 Pojednostavljeni PageRank algoritam

2.5.1 Osnovne ideje

Pretpostavimo da se Web sastoji od samo 4 čvora povezana hiperlinkovima kao što je prikazano na slici 2.3. Dakle, zasad ćemo zanemariti složenu strukturu i posmatrati jednostavan, jako povezan, graf.



Slika 2.3: Jednostavni graf Web-a sa 4 čvora

Kako bismo iskoristili hiperlinkove, jedna ideja je da stranice rangiramo prema broju linkova koji pokazuju na njih (eng. backlink). Međutim, mehanizam manipulacije je opet lako uočljiv i jednostavno primjenjiv: napraviti veliki broj beznačajnih stranica sa linkovima na stranicu čiji rang želimo povećati. Da bi se to prevazišlo, svakom linku se pridružuje vrijednost, a koja se dobija na osnovu ranga stranice na kojoj se nalazi link na način da se vrijednost ranga ravnomjerno distribuira na sve linkove. Na taj način postiže se da broj linkova nije presudan u bodovanju značaja stranice, obzirom da će stranica koja ima veliki rang, a samo jedan izlazni link, svoje bodove propagirati na tu stranicu, kojoj je onda dovoljan samo jedan ulazni link. Dakle, ideja je da se povjerenje prenosi. Međutim, izvjesno je da je definicija ranga rekurzivna, odnosno rang jedne stranice računa se na osnovu rangova drugih stranica, pa je pitanje kako su nam poznati ti rangovi. Ideja je da se svim stranicama na početku dodijeli proizvoljan rang, tako da je normalizirana suma svih rangova jedanaka 1, a zatim iterativno računaju rangovi, dok se vrijednosti ne stabiliziraju, pod uslovom da iterativni proces konvergira.

Da bismo rang jedne stranice distribuirali, neophodno je odrediti izlazne stepene po formuli 2.2.
Neka je $P = \{P_1, P_2, ..., P_n\}$ skup stranica (čvorova) koje su na neki način povezani hiperlinkovima (granama). Definisat ćemo skup B_X kao skup čvorova koji su spojeni sa čvorom P_X (grana je usmjerena ka čvoru P_X). Sada možemo definisati *rang* stranice P_X kao:

$$r(P_X) = \sum_{Y \in B_X} \frac{r(P_Y)}{d_g^+(P_Y)} \tag{2.13}$$

Iterativni proces za računanje ranga definisat ćemo kao:

$$r^{(k+1)}(P_X) = \sum_{Y \in B_X} \frac{r^{(k)}(P_Y)}{d_g^+(P_Y)}, \quad k = 0, 1, 2... \tag{2.14}$$

Uobičajeno je da se na početku svim stranicama dodijeli isti rang po formuli:

$$r^{(0)}(P_i) = \frac{1}{n}, \quad i = 1, 2, ..., n \tag{2.15}$$

gdje je n broj stranica (čvorova).
Opisani proces demonstrirat ćemo na grafu sa slike 2.3. Možemo uočiti na koji način se distribuira vrijednost ranga po granama (hiperlinkovima). Izlazni stepeni stranica $\{A, B, C, D\}$ su:

$$d_g^+(A) = 2$$
$$d_g^+(B) = 2$$
$$d_g^+(C) = 1$$
$$d_g^+(D) = 2$$

Početni rangovi (nulta iteracija):

$$r^{(0)}(A) = \frac{1}{4}, \quad r^{(0)}(B) = \frac{1}{4}, \quad r^{(0)}(C) = \frac{1}{4}, \quad r^{(0)}(D) = \frac{1}{4}$$

Prva iteracija:

$$r^{(1)}(A) = \frac{r^{(0)}(D)}{d_g^+(D)} = \frac{\frac{1}{4}}{2} = \frac{1}{8}$$
$$r^{(1)}(B) = \frac{r^{(0)}(A)}{d_g^+(A)} + \frac{r^{(0)}(D)}{d_g^+(D)} = \frac{\frac{1}{4}}{2} + \frac{\frac{1}{4}}{2} = \frac{1}{8} + \frac{1}{8} = \frac{1}{4}$$

$$r^{(1)}(C) = \frac{r^{(0)}(B)}{d_g^+(B)} = \frac{\frac{1}{4}}{2} = \frac{1}{8}$$

$$r^{(1)}(D) = \frac{r^{(0)}(A)}{d_g^+(A)} + \frac{r^{(0)}(B)}{d_g^+(B)} + \frac{r^{(0)}(C)}{d_g^+(C)} = \frac{\frac{1}{4}}{2} + \frac{\frac{1}{4}}{2} + \frac{\frac{1}{4}}{1} = \frac{1}{8} + \frac{1}{8} + \frac{1}{4} = \frac{1}{2}$$

Uočimo da je zbir svih rangova i dalje jednak 1.
Druga iteracija:

$$r^{(2)}(A) = \frac{r^{(1)}(D)}{d_g^+(D)} = \frac{\frac{1}{2}}{2} = \frac{1}{4}$$

$$r^{(2)}(B) = \frac{r^{(1)}(A)}{d_g^+(A)} + \frac{r^{(1)}(D)}{d_g^+(D)} = \frac{\frac{1}{8}}{2} + \frac{\frac{1}{2}}{2} = \frac{1}{16} + \frac{1}{4} = \frac{5}{16}$$

$$r^{(2)}(C) = \frac{r^{(1)}(B)}{d_g^+(B)} = \frac{\frac{1}{4}}{2} = \frac{1}{8}$$

$$r^{(2)}(D) = \frac{r^{(1)}(A)}{d_g^+(A)} + \frac{r^{(1)}(B)}{d_g^+(B)} + \frac{r^{(1)}(C)}{d_g^+(C)} = \frac{\frac{1}{8}}{2} + \frac{\frac{1}{4}}{2} + \frac{\frac{1}{8}}{1} = \frac{1}{16} + \frac{1}{8} + \frac{1}{8} = \frac{5}{16}$$

Zbir rangova nakon 2. iteracije iznosi $r^{(2)}(A) + r^{(2)}(B) + r^{(2)}(C) + r^{(2)}(D) = \frac{1}{4} + \frac{5}{16} + \frac{1}{8} + \frac{5}{16} = 1$

Istim postupkom, nakon 15. iteracija, rangovi se stabiliziraju na vrijednostima:

$$r^{(15)}(A) = \frac{4}{21}, \quad r^{(15)}(B) = \frac{6}{21}, \quad r^{(15)}(C) = \frac{3}{21}, \quad r^{(15)}(D) = \frac{8}{21}$$

2.5.2 Matrična reprezentacija

Za neki skup stranica $P = \{P_1, P_2, ..., P_n\}$ definisat ćemo kvadratnu matricu tranzicija $\mathbf{H}$ reda n kao:

$$h_{i,j} = \begin{cases} \frac{1}{d_g^+(P_j)} & \text{ako stranica } P_j \text{ pokazuje na stranicu } P_i, \\ 0 & \text{u suprotnom} \end{cases} \tag{2.16}$$

Rangove svih stranica predstavit ćemo kao vektor kolonu $\mathbf{r}$:

$$\begin{pmatrix} r(P_1) \\ r(P_2) \\ \vdots \\ r(P_n) \end{pmatrix}$$

Dakle $i-ta$ kolona predstavlja vrijednosti distribucije ranga stranice P_i na ostale stranice. Iterativni postupak 2.14 sada možemo predstaviti u matričnoj formi kao:

$$\mathbf{r}^{(k+1)} = \mathbf{H}\mathbf{r}^{(k)}, \quad k = 0, 1, 2, ... \tag{2.17}$$

Postupak ćemo primijeniti nekoliko puta:

$$\mathbf{r}^{(1)} = \mathbf{H}\mathbf{r}^{(0)}$$

$$\mathbf{r}^{(2)} = \mathbf{H}\mathbf{r}^{(1)} = \mathbf{H}\mathbf{H}\mathbf{r}^{(0)} = \mathbf{H}^2\mathbf{r}^{(0)}$$
$$\mathbf{r}^{(3)} = \mathbf{H}\mathbf{r}^{(2)} = \mathbf{H}\mathbf{H}^2\mathbf{r}^{(0)} = \mathbf{H}^3\mathbf{r}^{(0)}$$

Uočimo da iterativni proces 2.17 možemo zapisati u svojstvu potencije matrice tranzicija **H**:

$$\mathbf{r}^{(k)} = \mathbf{H}^k\mathbf{r}^{(0)}, \quad k = 1,2,3,... \tag{2.18}$$

Primijenimo postupak na primjer sa slike 2.3:

$$\mathbf{H} = \begin{array}{c} \begin{matrix} A & B & C & D \end{matrix} \\ \begin{pmatrix} 0 & 0 & 0 & \frac{1}{2} \\ \frac{1}{2} & 0 & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 1 & 0 \end{pmatrix} \end{array} \begin{matrix} A \\ B \\ C \\ D \end{matrix} \qquad \mathbf{r}^{(0)} = \begin{pmatrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{pmatrix}$$

$$\mathbf{r}^{(1)} = \mathbf{H}\mathbf{r}^{(0)} = \begin{pmatrix} 0 & 0 & 0 & \frac{1}{2} \\ \frac{1}{2} & 0 & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 1 & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{pmatrix} = \begin{pmatrix} \frac{1}{8} \\ \frac{1}{4} \\ \frac{1}{8} \\ \frac{1}{2} \end{pmatrix}$$

$$\mathbf{r}^{(2)} = \mathbf{H}\mathbf{r}^{(1)} = \begin{pmatrix} 0 & 0 & 0 & \frac{1}{2} \\ \frac{1}{2} & 0 & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 1 & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{8} \\ \frac{1}{4} \\ \frac{1}{8} \\ \frac{1}{2} \end{pmatrix} = \begin{pmatrix} \frac{1}{4} \\ \frac{5}{16} \\ \frac{1}{8} \\ \frac{5}{16} \end{pmatrix}$$

ili:

$$\mathbf{r}^{(2)} = \mathbf{H}^2\mathbf{r}^{(0)} = \begin{pmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{2} & 0 \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{2} & \frac{1}{4} \\ \frac{1}{4} & 0 & 0 & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{2} & 0 & \frac{1}{2} \end{pmatrix} \begin{pmatrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{pmatrix} = \begin{pmatrix} \frac{1}{4} \\ \frac{5}{16} \\ \frac{1}{8} \\ \frac{5}{16} \end{pmatrix}$$

Opisani postupak poznat je kao *metod potencija*, a predstavlja iterativni postupak za pronalaženje najveće svojstvene vrijednosti po modulu (dominantne svojstvene vrijednosti) i pripadnog svojstvenog vektora

2.5.3 Tumačenje ranga iz aspekta vjerovatnoće

Nakon što smo strukturu Web-a predstavili grafom, i pridružili mu odgovarajuću matricu tranzicija **H**, rangu stranice možemo dodijeliti još jedno tumačenje. Kako je rang mjera relevantnosti

(značaja, popularnosti), tu vrijednost možemo posmatrati i kao vjerovatnoću da će slučajni korisnik krećući se kroz Web graf dospjeti na tu stranicu. Vrijednostima izlaznih linkova, odnosno elementima matrice **H**, na primjeru sa slike 2.3, možemo dati sljedeće vjerovatnosno značenje: Ako se slučajni korisnik nalazi na stranici D postoji vjerovatnoća $\frac{1}{2}$ da će posjetiti stranicu A, te vjerovatnoća $\frac{1}{2}$ da će otići na stranicu B. Inicijalno, sve stranice imaju istu vjerovatnoću da budu izabrane za početnu tačku obilaska grafa, pa je bazna distribucija $\mathbf{r}^{(0)} = [\frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}]^T$. Vjerovatnoće da će korisnik posjetiti neku stranicu nakon k koraka, date su vektorom $\mathbf{H}^k\mathbf{r}^{(0)}$. Iz načina na koji smo kreirali matricu **H** jasno je da je zbir u svakoj koloni jednak 1, te da su svi elementi nenegativni, stoga je matrica **H** *stohastička po kolonama*. Osim toga, skup svih stranica Web-a je konačan, pri čemu prelazak na drugu stranicu ovisi isključivo od trenutne stranice. Obzirom da rang posmatramo u diskretnim trenucima, zaključujemo da *PageRank* zapravo predstavlja diskretni stohastički proces, preciznije *Markovljev lanac*. Sekvenca $\mathbf{Hr}^{(0)}$, $\mathbf{H}^2\mathbf{r}^{(0)}$, $\mathbf{H}^3\mathbf{r}^{(0)}$ ...je sekvenca vjerovatnosnih distribucija slučajnih varijabli, a čije distribucije predstavljaju trenutni rang stranica, te pod uslovima opisanim u 2.3.2 teže stacionarnoj distribuciji π^*, odnosno traženom rangu. U nastavku ćemo vektor rangova označavati sa π kako bismo naglasili da se radi o stacionarnoj distribuciji.

* * *

Prilikom definisanja modela predstavljenog u ovom poglavlju, napravili smo dvije pretpostavke zbog kojih isti nije univerzalan, te se ne može primijeniti na realnu strukturu:

- Apstrahovali smo Web kao jako povezan graf, zbog čega su mogući obilasci i šetnje, bez izoliranih ili apsorbirajućih čvorova (nepovezane i izlazne komponente), a za koje mnogi izvedeni zaključci ne vrijede bez modifikacija (uključujući konvergenciju).
- Popularnost, odnosno vjerovatnoća posjete određene stranice bazirala se na ulaznim linkovima, odnosno na praćenju linkova sa drugih stranica. To bi značilo da se stranica koja nema ulaznih linkova (IN komponente), ne može nikad posjetiti, a što je neispravno obzirom da uvijek postoji šansa da će korisnik otići na neku nepovezanu stranicu ("skočiti").

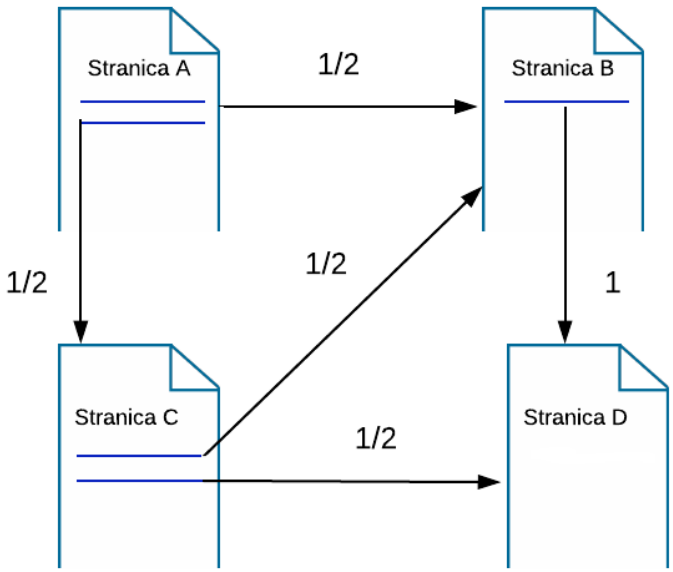
U sljedećoj sekciji obrazložit ćemo kako se prethodno prikazani model može učiniti upotrebljivim uvođenjem elemenata koji će odstraniti opisana ograničenja.

2.6 Generalizirani PageRank algoritam

U ovoj sekciji razmotrit ćemo načine prevazilaženja problema opisanih u prethodnom poglavlju, te pokazati kako možemo biti sigurni u valjanost i kovergenciju rješenja.

2.6.1 Apsorbirajući čvorovi

Web je heterogen i sastoji se od mnogo komponenti povezanosti što smo pokazali u 2.4. Na primjeru dva jednostavna grafa vidjet ćemo kako se to manifestira na naš model.



Slika 2.4: Graf sa apsorbirajućim čvorom

Graf sa slike 2.4 posjeduje jedan apsorbirajući/viseći čvor (engl. dangling node) te, posljedično, nije jako povezan, odnosno nije ireducibilan, što znači da uvjet koji garantira postojanje stacionarne distribucije nije ispunjen. Da bismo se uvjerili kako se to odražava na traženje ranga u ovoj situaciji, na ovaj graf primijenit ćemo iterativni postupak 2.17:

$$\mathbf{r}^{(1)} = \mathbf{H}\mathbf{r}^{(0)} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & 0 & 0 \\ 0 & 1 & \frac{1}{2} & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \\ \frac{1}{4} \end{pmatrix} = \begin{pmatrix} 0 \\ \frac{1}{4} \\ \frac{1}{8} \\ \frac{3}{8} \end{pmatrix}$$

$$\mathbf{r}^{(2)} = \mathbf{H}\mathbf{r}^{(1)} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & 0 & 0 \\ 0 & 1 & \frac{1}{2} & 0 \end{pmatrix} \begin{pmatrix} 0 \\ \frac{1}{4} \\ \frac{1}{8} \\ \frac{3}{8} \end{pmatrix} = \begin{pmatrix} 0 \\ \frac{1}{16} \\ 0 \\ \frac{5}{16} \end{pmatrix}$$

$$\vdots$$

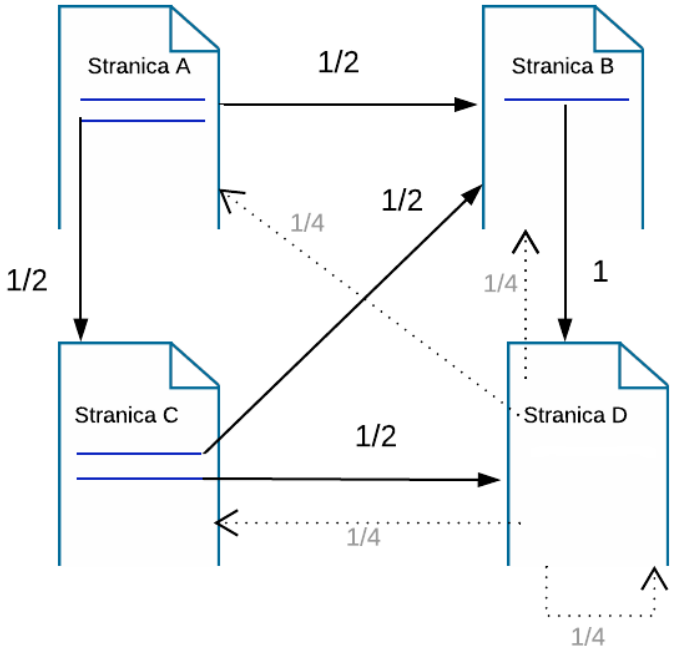
$$\mathbf{r}^{(4)} = \mathbf{H}\mathbf{r}^{(3)} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & 0 & 0 \\ 0 & 1 & \frac{1}{2} & 0 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 0 \\ \frac{1}{16} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Čvor D je postepeno apsorbirao rangove drugih stranica, te su na kraju rangovi svih stranica 0. Takav ishod je kontraintuitivan, obzirom da stranica D ima dva ulazna linka, pa je očekivano da ima neki značaj. Dakle, u ovom slučaju, iterativni proces 2.17 ne konvergira ka rješenju. Matrica $\mathbf{H}$ ima jednu svojstvenu vrijednost $\lambda = 0$ višestrukosti 4, pri čemu ne postoji pozitivan svojstveni vektor.

Nule u koloni koja odgovora stranici D matrice tranzicija $\mathbf{H}$, znače da, kada korisnik jednom dospije na stranicu D, vjerovatnoća da će preći na bilo koju drugu iznosi 0. Istina je, zapravo, da iz tog čvora, postoji ista vjerovatnoća prelaska u bilo koji drugi čvor, uključujući i njega samog (dakle ostanak na istoj stranici). Rješenje opisanog problema je da se elementi vrijednosti 0 zamijene elementima $\frac{1}{n}$ gdje je n broj čvorova. Stoga ćemo modificirati formulu za kreiranje matrice tranzicija. Modificiranu matricu označavat ćemo sa $\mathbf{H}'$:

$$h'_{i,j} = \begin{cases} \frac{1}{d_g^+(P_j)} & \text{ako stranica } P_j \text{ pokazuje na stranicu } P_i, \\ \frac{1}{n} & \text{ako je stranica } P_j \text{ apsorbirajuća,} \\ 0 & \text{u suprotnom} \end{cases} \tag{2.19}$$

Graf 2.4 sada postaje jako povezan:



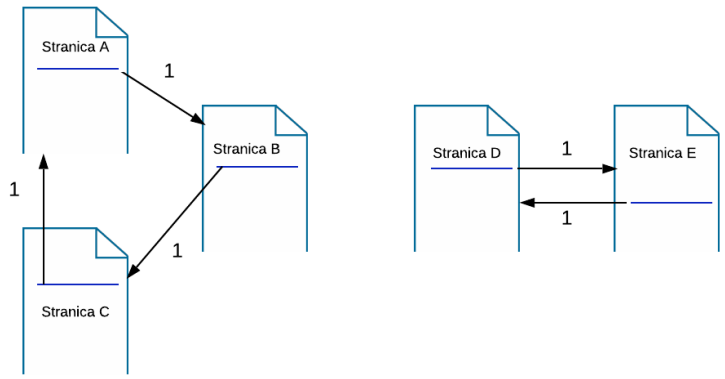
Slika 2.5: Modificirani graf sa apsorbirajućim čvorom

Ako opet primijenimo postupak 2.17 nad modificiranom matricom tranzicija (koji ovdje nećemo prikazivati), isti će konvergirat ka rješenju, te stabilizirati se u 15. iteraciji, čime dobijamo finalne vrijednosti rangova:

$$\mathbf{r}^{(15)} = \begin{pmatrix} \frac{4}{35} \\ \frac{9}{35} \\ \frac{6}{35} \\ \frac{16}{35} \end{pmatrix}$$

2.6.2 Nepovezane komponente

Osim apsorbirajućih čvorova, diverzitet Web-a je i u tome što posjeduje ogroman broj komponenti koje nisu ni na koji način povezane sa ostatkom. Na slici 2.6 prikazan je jednostavni graf Web-a sa dvije komponente povezanosti.

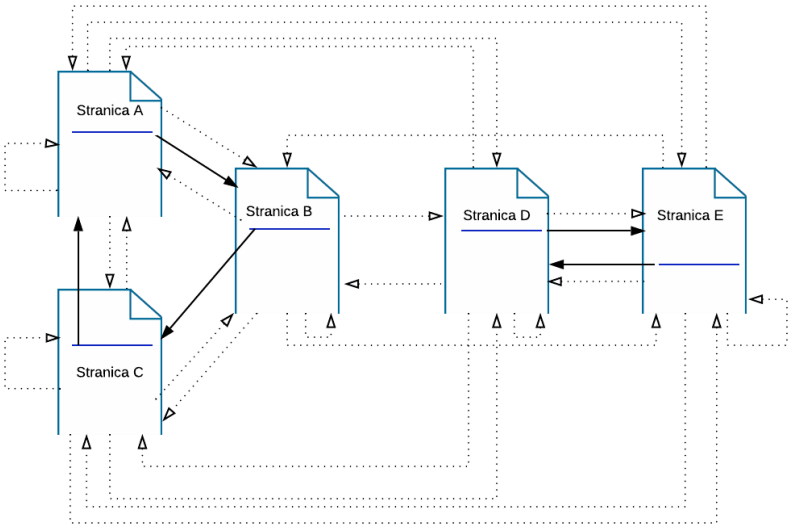


Slika 2.6: Graf sa dvije komponente povezanosti

Ukoliko postoji više komponenti povezanosti, za razliku od apsorbirajućih čvorova, moguće je odrediti finalni rang **r**, ali je on višeznačan. Razlog tome je što stranice jedne komponente nemaju utjecaj na rang stranica drugih komponenti, jer se distribucija vrši unutar komponente. Na taj način, ako bismo željeli održati osobinu o normaliziranoj sumi (suma svih rangova 1), tada bismo određenu količinu ranga dodijelili svakoj komponenti, a nakon toga bi se isti raspodijelio na pojedine stranice.

Na primjeru sa slike 2.6, u skladu sa ovim rezonom, finalni rang može biti $\mathbf{r} = [\frac{1}{3}, \frac{1}{3}, \frac{1}{3}, 0, 0]^T$ ili $\mathbf{r} = [0, 0, 0\ \frac{1}{2}, \frac{1}{2}]^T$. To nam potvrđuje i činjenica da matrica tranzicija ovog grafa ima svojstvenu vrijednost $\lambda = 1$, ali višestrukosti 2, a kojima su pridružena dva navedena vektora. U ovom slučaju, graf također nije ireducibilan, ali postoje pozitivni svojstveni vektori. Obzirom da model kojemu ne možemo jednoznačno odrediti distribuciju nije upotrebljiv, nad grafom ćemo izvršiti neophodne modifikacije.

Da bismo u model uključili i mogućnost prelaska iz komponente u komponentu, odnosno "skok" sa bilo koje stranice na bilo koju drugu, na usmjereni graf dodat ćemo grane iz svakog čvora na sve druge bez obzira da li već postoje, čime će graf postati jako povezan.



Slika 2.7: Proširenje do jako povezanog grafa

Efektivno to znači da ćemo na matricu tranzicija $\mathbf{H}'$ dodati matricu iste dimenzionalnosti čiji su svi elementi $\frac{1}{n}$, gdje je n broj čvorova. Tu matricu ćemo označavati sa $\frac{1}{n}\mathbf{E}$, gdje je $\mathbf{E}$ matrica dimenzija $n \times n$ čiji su svi elementi jedinice.

$$\mathbf{H}' + \frac{1}{5}\mathbf{E} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix} + \begin{pmatrix} \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \\ \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \\ \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \\ \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \\ \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \end{pmatrix}$$

Novodobijeni graf je ireducibilan i aperiodičan, međutim odgovarajuća matrica više nije *stohastička po kolonama*, obzirom da je zbir vjerovatnoća iz svakog čvora $2 > 1$, stoga ćemo u nastavku razmotriti normalizaciju modela.

Faktor prigušenja d

Zbir elemenata svake kolone matrice $\mathbf{H}' + \frac{1}{n}\mathbf{E}$ je 2, te ćemo s tim u vezi svaku od matrica pomnožiti nekim koeficijentom, tako da zbirna matrica postane stohastička. Neka su to α i β, pa je nova matrica: $\alpha\mathbf{H}' + \frac{\beta}{n}\mathbf{E}$. Kako je zbir svih elemenata bilo koje kolone obje matrice jednak 1, jasno je da se uslov da zbir elemenata svake kolone zbirne matrice svodi na:

$$\alpha \cdot 1 + \beta \cdot 1 = 1 \implies \alpha + \beta = 1 \implies \beta = 1 - \alpha$$

Dakle, koeficijenti su α i $1-\alpha$. Larry Page i Sergey Brin su empirijski odredili da je najpogodnija vrijednost $\alpha = 0.85$ [1], a obično se označava sa d od engleskog *damping factor*.
Konačno, dolazimo do definicije koju Larry Page i Sergey Brin predlažu u [1]:

Definicija 2.6.1 *Neka su $T_1, T_2, ..., T_n$ stranice koje pokazuje na stranicu A. Parametar d je faktor prigušenja i poprima vrijednost između 0 i 1, najčešće 0.85. Neka je $C(A)$ broj izlaznih linkova stranice A. Rang stranice A (engl. PageRank) je onda:*

$$PR(A) = (1-d) + d\left(\frac{PR(T_1)}{C(T_1)} + \frac{PR(T_2)}{C(T_2)} + ... + \frac{PR(T_n)}{C(T_n)}\right). \tag{2.20}$$

Međutim, jasno je da ovako definirani rangovi ne čine vjerovatnosnu distribuciju nad skupom stranica obzirom da prvi sabirak nije distribuiran po stranicama nego je konstantan za sve skupove stranica i iznosi 0.15. Dakle, normalizirani rang stranice A iznosi:

$$PR(A) = \frac{1-d}{n} + d(\frac{PR(T_1)}{C(T_1)} + \frac{PR(T_2)}{C(T_2)} + ... + \frac{PR(T_n)}{C(T_n)}). \tag{2.21}$$

Google matrica

Nakon što smo uveli faktor slučajnosti (matrica $\mathbf{E}$) i faktor prigušenja d, definisat ćemo i Google matricu za skup od n stranica kao:

$$\mathbf{G} = (1-d)\frac{1}{n}\mathbf{E} + d\mathbf{H}'. \tag{2.22}$$

Google matrica zadovoljava sve uvjete za postojanje jedinstvene svojstvene vrijednosti $\lambda = 1$, stoga se ukazuje izravan, ali naivan pristup za pronalazak rangova: pronaći svojstveni vektor koji odgovara svojstvenoj vrijednosti $\lambda = 1$. Dakle, vrijedi sljedeća jednačina:

$$\pi = \mathbf{G}\pi, \tag{2.23}$$

gdje π predstavlja stacionarnu distribuciju, odnosno vektor kolonu rangova svih stranica.

Većina algoritama koji bi nam omogućili pronalazak svojstvenih vrijednosti ima vremensku kompleksnost $O(n^3)$, ili nešto manju za posebne oblike matrica. Drugi naivan pristup je direktno rješavanje sistema linearnih jednačina. Naime jednačinu 2.23 možemo posmatrati kao sistem linearnih jednačina s nepoznatim vektorom π, međutim poznati algoritmi za direktno rješavanje takvih problemea su također kompleksnosti cca $O(n^3)$. Da bi se izvršio algoritam kompleksnosti $O(n^3)$ na ulazu reda 10^6 pod pretpostavkom da računar na kojem se izvodi algoritam može obaviti 10 operacija u mikrosekundi, potrebno je više od 3170 godina [11]. Kako se današnji Web sastoji od više milijardi stranica, jasno je da pristup koji se zasniva na direktnom pronalasku rješenja (bilo svojstvenih vrijednosti bilo rješenja linearnog sistema jednačina) nije upotrebljiv, već da je neophodno primijeniti neki od iterativnih postupaka.

Demonstracije radi, na primjeru sa slike 2.7 prikazat ćemo oba načina.

- Svojstvene vrijednosti i svojstveni vektori

  Svojstvene vrijednosti matrice $\mathbf{G}$ su: 1, 0.85, -0.85, -0.4250 + 0.7361i, +0.4250 - 0.7361i. Normalizirani svojstveni vektor koji odgovara svojstvenoj vrijednosti $\lambda = 1$, odnosno vektor konačnih rangova stranica je $[\frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}, \frac{1}{5}]^T$

- Sistem jednačina

  Neka je $\pi = [p(A), p(B), p(C), p(D), p(E)]^T$. Iz sistema $\mathbf{G}\pi = \pi$ slijedi:

$$\begin{pmatrix} \frac{3}{10} & \frac{3}{10} & \frac{88}{100} & \frac{3}{100} & \frac{3}{100} \\ \frac{88}{100} & \frac{3}{100} & \frac{3}{100} & \frac{3}{100} & \frac{3}{100} \\ \frac{3}{100} & \frac{88}{100} & \frac{3}{100} & \frac{3}{100} & \frac{3}{100} \\ \frac{3}{100} & \frac{3}{100} & \frac{3}{100} & \frac{3}{100} & \frac{88}{100} \\ \frac{3}{100} & \frac{3}{100} & \frac{3}{100} & \frac{88}{1000} & \frac{3}{100} \end{pmatrix} \begin{pmatrix} p(A) \\ p(B) \\ p(C) \\ p(D) \\ p(E) \end{pmatrix} = \begin{pmatrix} p(A) \\ p(B) \\ p(C) \\ p(D) \\ p(E) \end{pmatrix}$$

Nakon množenja matrice i vektora te oduzimanja desne strane dobijamo homogeni sistem:

$$\begin{gathered}
-\frac{97}{100}p(A)+\frac{3}{100}p(B)+\frac{88}{100}p(C)+\frac{3}{100}p(D)+\frac{3}{100}p(E)=0\\
\frac{88}{100}p(A)-\frac{97}{100}p(B)+\frac{3}{100}p(C)+\frac{3}{100}p(D)+\frac{3}{100}p(E)=0\\
\frac{3}{100}p(A)+\frac{88}{100}p(B)-\frac{97}{100}p(C)+\frac{3}{100}p(D)+\frac{3}{100}p(E)=0\\
\frac{3}{100}p(A)+\frac{3}{100}p(B)+\frac{3}{100}p(C)-\frac{97}{100}p(D)+\frac{88}{100}p(E)=0\\
\frac{3}{100}p(A)+\frac{3}{100}p(B)+\frac{3}{100}p(C)+\frac{88}{1000}p(D)-\frac{97}{100}p(E)=0
\end{gathered}$$

Da bi sistem imao i netrivijalno rješenje, potrebno je i dovoljno da matrica ima rang manji od broja promjenljivih. Matrica iz primjera ima rang 4, što znači da ima beskonačno mnogo rješenja. Rješenje glasi $[t,t,t,t,t]$, $t \in \mathbb{R}$. Međutim, nama je potrebno ono rješenje u kojem zbir svih elemenata iznosi 1, stoga na postojeći sistem dodajemo još jednu jednačinu:

$$p(A)+p(B)+p(C)+p(D)+p(E)=1.$$

Odnosno:

$$\begin{gathered}
t+t+t+t+t=1\\
5t=1\\
t=\frac{1}{5}
\end{gathered}$$

Dakle, konačno rješenje je $\pi=[\frac{1}{5},\frac{1}{5},\frac{1}{5},\frac{1}{5},\frac{1}{5}]^T$

Obzirom da se radi o Markovljevom lancu, poznatiji i upotrebljiviji metod je spomenuti *metod potencija*:

$$\pi^{(k)}=\mathbf{G}\pi^{(k-1)}, \quad k=1,2,3... \tag{2.24}$$

odnosno

$$\pi^{(k)}=\mathbf{G}^k\pi^{(0)}, \tag{2.25}$$

gdje vidimo i zbog čega se metod naziva metod potencija: rang u $k-toj$ iteraciji jednak je umnošku $k-te$ potencije matrice $\mathbf{G}$ i početne distribucije.

Intuicija iza konvergencije metoda potencija je u sljedećem. Neka je $\{1,\lambda_2,\lambda_3...\lambda_n\}$ spektar matrice $\mathbf{G}$ i neka su $\{\mathbf{v}_1,\mathbf{v}_2,\mathbf{v}_3...\mathbf{v}_n\}$ odgovarajući svojstveni vektori. Jednostavnosti radi, pretpostavimo da početna distribucija $x^{(0)}$ leži u potprostoru koji razapinju svojstveni vektori matrice $\mathbf{G}$. Tada $x^{(0)}$ možemo napisati kao linearnu kombinaciju svojstvenih vektora matrice $\mathbf{G}$:

$$x^{(0)}=\mathbf{v}_1+\alpha_2\mathbf{v}_2+...+\alpha_n\mathbf{v}_n \tag{2.26}$$

Nadalje, izvršit ćemo lijevo množenje obje strane matricom $\mathbf{G}$. Iz $\pi^{(1)}=\mathbf{G}\pi^{(0)}$ i $\mathbf{G}\mathbf{v_i}=\lambda_i\mathbf{v_i}$:

$$x^{(1)}=\mathbf{G}x^{(0)}=\mathbf{G}\mathbf{v}_1+\alpha_2\mathbf{G}\mathbf{v}_2+...+\alpha_n\mathbf{G}\mathbf{v}_n=\lambda_1\mathbf{v}_1+\alpha_2\lambda_2\mathbf{v}_2+...+\alpha_n\lambda_n\mathbf{v}_n \tag{2.27}$$

Kako je dominantna svojstvena vrijednost $\lambda_1 = 1$, slijedi:

$$x^{(1)} = \mathbf{v}_1 + \alpha_2\lambda_2\mathbf{v}_2 + ... + \alpha_n\lambda_n\mathbf{v}_n \tag{2.28}$$

Ako isti postupak ponovimo k puta, dobit ćemo:

$$x^{(k)} = \mathbf{v}_1 + \alpha_2\lambda_2^k\mathbf{v}_2 + ... + \alpha_n\lambda_n^k\mathbf{v}_n \tag{2.29}$$

Iz pretpostavke o jedinstvenosti svih svojstvenih vrijednosti slijedi $1 > |\lambda_2| > |\lambda_3| > ... > |\lambda_n|$. Dakle, $\forall\ i = 2,3,...,n$ vrijedi $\lambda_i^k \to 0$, što znači da vektor $x^{(k)} \to \mathbf{v}_1$, odnosno $x^{(k)} \to \pi$, što je trebalo pokazati.

* * *

U ovom poglavlju osvrnuli smo se na teoretsku pozadinu PageRank algoritma što uključuje svojstva koja garantuju postojanje jedinstvenog rješenja, te svođenje PageRank problema na dva, već poznata, problema: pronalaženje svojstvenih vrijednosti i svojstvenih vektora te rješavanje sistema linearnih jednačina. U sljedećem poglavlju razmotrit ćemo implementacijske izazove ovih koncepata.

Poglavlje 3

Implementacija

3.1 Metod potencija

Nakon svih modifikacija kako bi se postigla ireducibilnost i aperiodičnost, matrica tranzicija je postala *pozitivna*, pa se čini da bismo je u memoriji morali pamtiti u cjelosti. Međutim, nulti elementi koji su nakon modifikacija postali pozitivni, sadrže konstantnu vrijednost poznatu prije početka izvršavanja algoritma, pa je pitanje možemo li nekako smanjiti memorijsko zauzeće. S tim u vezi, posmatrat ćemo iterativni postupak 2.24:

$$\pi^{(k+1)} = \mathbf{G}\pi^{(k)}$$
$$\pi^{(k+1)} = [(1-d)\tfrac{1}{n}\mathbf{E} + d\mathbf{H}']\pi^{(k)}$$
$$\pi^{(k+1)} = (1-d)\tfrac{1}{n}\mathbf{E}\pi^{(k)} + d\mathbf{H}'\pi^{(k)}, \quad k = 0,1,2...$$

Obzirom da je *problematičan* sabirak sa matricom $\mathbf{E}$, fokusirat ćemo se na taj dio sume. Matrica $\mathbf{E}$ je dimenzija $n \times n$, gdje je n broj čvorova, dok je π dimenzionalnosti $n \times 1$, što znači da je rezultujući vektor također dimenzija $n \times 1$.

$$\mathbf{E}\pi = \begin{pmatrix} 1 & 1 & \dots \\ \vdots & \ddots & \\ 1 & & 1 \end{pmatrix} \begin{pmatrix} p(1) \\ \vdots \\ p(n) \end{pmatrix} = \begin{pmatrix} p(1) + (p2) + \dots + p(n) \\ \vdots \\ p(1) + (p2) + \dots + p(n) \end{pmatrix}$$

Kako π predstavlja stacionarnu distribuciju, suma elemenata vektora je 1, te je stoga rezultujući vektor, zapravo vektor kolona sačinjen od jedinica:

$$\mathbf{E}\pi = \mathbf{e}$$

Otuda dolazimo do nove formule iterativnog postupka:

$$\pi^{(k+1)} = (1-d)\frac{1}{n}\mathbf{e} + d\mathbf{H}'\pi^{(k)}, \quad k = 0,1,2... \tag{3.1}$$

Međutim, matrica $\mathbf{H}'$ sadrži kolone koje odgovaraju apsorbirajućim čvorovima sa elementima koji iznose $\frac{1}{n}$. Obzirom da 80% Web-a čine upravo apsorbirajući čvorovi, bilo bi izuzetno neefikasno pamtiti vrijednosti tih, vještački dodanih, linkova (radi se o milijardama hiperlinkova), kada je poznato da se rang apsorbirajućeg čvora uniformno raspoređuje, te ćemo stoga dodatno transformisati formulu 3.34.

Neka je vektor **a** definisan na sljedeći način:

$$a_i = \begin{cases} 1 & \text{ako je stranica } P_i \text{ apsorbirajuća,} \\ 0 & \text{u suprotnom} \end{cases} \tag{3.2}$$

Sada matricu H' možemo zapisati kao: $\mathbf{H}' = \mathbf{H} + \frac{1}{n}\mathbf{e}\mathbf{a}^T$, te uvrstiti u formulu 3.34:

$$\pi^{(k+1)} = (1-d)\frac{1}{n}\mathbf{e} + d\frac{1}{n}\mathbf{e}\mathbf{a}^T\pi^{(k)} + d\mathbf{H}\pi^{(k)}, \quad k = 0,1,2... \tag{3.3}$$

Prvi sabirak je poznat prije početka izvršavanja algoritma, a ostaje konstantan u svim iteracijama i za sve stranice. Drugi sabirak je vektor čiji elementi imaju vrijednost zbira rangova apsorbirajućih stranica pomnoženog sa uniformnom distribucijom $\frac{1}{n}$. Taj vektor predstavlja doprinos apsorbirajućih stranica rangovima ostalih stranica. Obzirom da je taj doprinos isti za sve stranice, u svakoj iteraciji računat ćemo ga samo jednom, te zatim dodavati na rang svake stranice. Na ovaj način ostvarili smo značajnu memorijsku i vremensku uštedu, obzirom da će se sav račun vršiti samo nad stvarnim hiperlinkovima.

Za implementaciju koristit ćemo programski jezik C++ obzirom da je zbog efikasnosti većina Google-a napisana u istom. Ulaz u algoritam predstavlja datoteka koja sadrži informacije o stranicama i hiperlinkovima, pri čemu su stranice predstavljene prirodnim brojevima, a notacija u datoteci je:

$$k : x, y, z$$

što označava da stranica k sadrži hiperlinkove ka stranicima x, y, z, obzirom da su te informacije poznate nakon što Web agent posjeti i zapamti neku stranicu sa svim neophodnim meta podacima. Da bismo testirali algoritam na različitim testnim skupovima, implementiran je generator datoteka opisane strukture.

Obzirom da posjedujemo ograničene količine RAM-a, cilj je raspoloživi prostor iskoristiti što racionalnije, a pamćenje cijele matrice tranzicije je prilično memorijski zahtjevno, pogotovo nakon što smo pokazali da je moguće koristiti samo dio informacija iz matrice tokom izvršavanja, a ostalo izračunati na početku. Nije zanemariva ni vremenska kompleksnost u tom slučaju, obzirom da bi se iteracija svodila na množenje matrice vektorom, a što je operacija kompleksnosti $O(n^2)$

Ako pogledamo formulu 2.21 uočit ćemo da je neophodno imati na raspolaganju informacije o tome koje stranice pokazuju na stranicu čiji rang se računa. Ukupan broj stranica može biti jako velik, te nije efikasno praviti više prolaza kroz datoteku da bi se saznao broj ulaznih linkova, a onda inicijalizirala neka struktura podataka fiksne veličine. Umjesto toga, potrebna nam je struktura podataka čija se veličina dinamički mijenja uz efikasno dodavanje elemenata, tako da u jednom prolazu možemo prikupiti informacije o ulaznim linkovima svih stranica. Paralelno ćemo pamtiti i vrijednost distribucije ranga stranice prema ostalima, odnosno $\frac{1}{d_g^+}$. To bi bilo moguće pamtiti u paru zajedno sa izlaznim linkom kome je pridružena ta distribucija, međutim taj broj je isti za sve izlazne linkove neke stranice, pa je jednostavnije u jednom nizu veličine broja stranica zapamtiti tu vrijednost, te istu koristiti za sve izlazne linkove.

Za pohranu ulaznih linkova koristit ćemo jednostruko povezanu listu. Lista je implementirana kroz klasu LinkedList, a od atributa sadrži početni i krajnji čvor liste, te dvije metode: za

dodavanje novog čvora, i metodu koja množi dva vektora, ali samo elemente na indeksima koji su vrijednosti čvorova liste.

Vektor π je dinamički alociran niz brojeva u pokretnom zarezu jednostruke preciznosti (float) po uzoru na originalni rad[1]. Najmanja vrijednost reprezentabilna u tom formatu je 1.175494351e−38, a u ovom radu koristit ćemo skupove veličine do 5 miliona čvorova što znači da je najmanji rang reda 10^{-8}, pa je jednostruka preciznost dovoljna za ovakvu primjenu. Kako bismo što više umanjili akumulaciju grešaka, međurezultate pamtit ćemo u formatu dvostruke preciznosti (double).

U dinamički alociranim nizovima cijelih brojeva i realnih brojeva jednostruke preciznosti pamtit ćemo indekse apsorbirajućih čvorova i vrijednosti $\frac{1}{d_G^+}$ za svaki čvor, respektivno.
Ti nizovi su enkapsulirani u klasu PageRank, zajedno sa još nekoliko atributa:

- numberOfPages(int) - ukupan broj stranica
- numOfDanglingNodes(int) - broj apsorbirajućih čvorova
- dampingFactor(float) - faktor prigušenja d
- eps(float) - tolerancija tj. maksimalna vrijednost odstupanja između vrijednosti dvije iteracija koja se toleriše da bi algoritam terminirao
- maxNumberOfIterations(int) - maksimalan dozvoljeni broj iteracija

Posljednja tri parametra imaju podrazumijevane vrijednosti 0.85, $1e^{-12}$ i 100, ako se drugačije ne specificira preko pristupnih metoda. Ispod je prikazan programski kod metode koja računa PageRank skupa stranica koji je učitan prilikom kreiranja instance klase PageRank.

Program 3.1: Metoda za računanje PageRank-a

```
1  void PageRank::calculateSimplePageRank() {
2      float* previousPageRanks = new float[numberOfPages]{};
3      double randomWalkProb = (1 - dampingFactor)/numberOfPages, maxError
           = 1;
4
5      while(maxError > eps && iterations < maxNumberOfIterations){
6          double danglingNodesAndRandomWalk =  dampingFactor*
               getDanglingNodesContribution() + randomWalkProb;
7          std::copy(pageRanks, pageRanks + numberOfPages,
               previousPageRanks);
8          maxError = 0;
9          for(int i = 0; i < numberOfPages; i++){
10             pageRanks[i] = dampingFactor * pLinkedLists[i].
                   multiplyDistributionWithBacklinks(outerLinkDistributions,
                    previousPageRanks) + danglingNodesAndRandomWalk;
11
12             double pageError = abs(previousPageRanks[i] - pageRanks[i]);
13             if(pageError > maxError ) maxError = pageError;
14         }
15         iterations++;
16     }
17     delete[] previousPageRanks;
18 }
```

Niz *previousPageRanks* predstavlja privremeni spremnik za rangove iz prethodne iteracije, na osnovu kojih se računaju rangovi u trenutnoj iteraciji, kao i odstupanja vrijednosti nove u odnosu na prethodnu iteraciju.

U trećoj liniji se inicijalizira sabirak koji je konstantan tokom svih iteracija, a u šestoj tu vrijednost sabiramo sa doprinosom apsorbirajućih čvorova ukupnom rangu, a koji je u jednoj iteraciji isti za sve čvorove.

U *for* petlji se računa novi rang za svaku stranicu pomoću metode klase LinkedList u kojoj se sabiraju umnošci distribucija stranica koje upućuju na stranicu čiji se rang računa i njihovog ranga iz prethodne iteracije. Programski kod metode **multiplyDistributionWithBacklinks** prikazan je ispod:

Program 3.2: Metoda za računanje doprinosa ulaznih linkova

```
1 double LinkedList::multiplyDistributionWithBacklinks(const float*
     probDistribution, const float* pageRanks) {
2     double sum = 0;
3     Node* tmp = head;
4     while(tmp){
5         sum += (probDistribution[tmp->link])*pageRanks[tmp -> link];
6         tmp = tmp->next;
7     }
8     return sum;
9 }
```

Dakle u ovoj metodi računamo sumu koju bismo u slučaju reprezentacije grafa matricom dobili množenjem jednog reda matrice vektorom rangova. Međutim, računanje ranga jedne stranice bi u tom slučaju bilo kompleksnosti $O(n)$, odnosno za sve stranice $O(n^2)$, neovisno o broju stvarnih ulaznih linkova. S druge strane, pohranjivanjem isključivo stvarnih linkova, postižemo da je kompleksnost računanja ranga jedne stranice $O(d_G^-)$, odnosno broj ulaznih linkova pri čemu je d_G^- obično značajno manje od n, a za sve stranice $O(m)$, gdje je m ukupan broj grana grafa.

Na kraju, prikazat ćemo metodu koja računa doprinos apsorbirajućih čvorova rangovima drugih stranica:

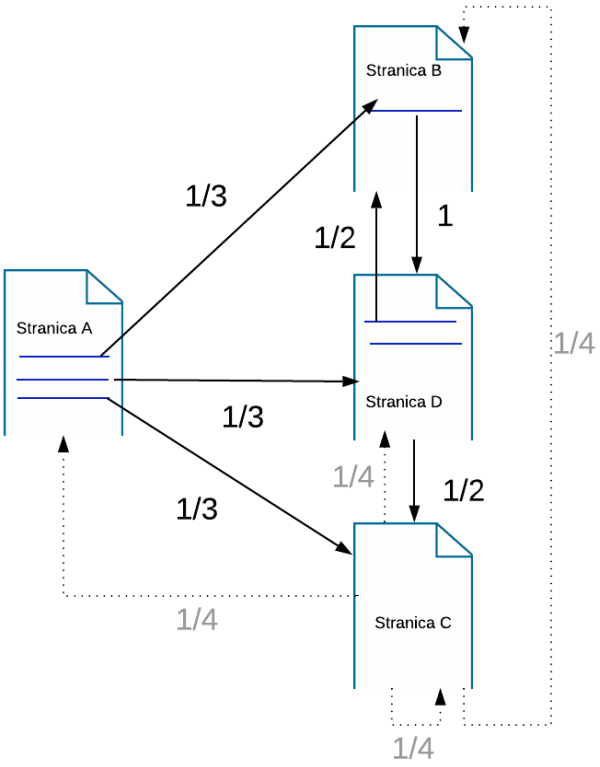
Program 3.3: Metoda za računanje doprinosa apsorbirajućih čvorova

```
1 double PageRank::getDanglingNodesContribution() {
2     double sum = 0;
3     for(int i = 0; i < numberOfDanglingNodes; i++){
4         sum += pageRanks[danglingNodes[i]];
5     }
6     return sum/numberOfPages;
7 }
```

Svi apsorbirajući čvorovi uniformno distribuiraju rang ostalim stranicama, odnosno svaka stranica dobija $\frac{1}{n}$ vrijednosti ranga jednog apsorbirajućeg čvora. Obzirom da je ta vrijednost ista za sve apsorbirajuće čvorove, kako bismo uštedili po jedno dijeljenje u iteraciji, ukupnu sumu rangova svih apsorbirajućih čvorova na kraju dijelimo sa ukupnim brojem stranica, čime dobijamo sabirak $\frac{1}{n}\mathbf{e}\mathbf{a}^T$ iz jednačine 3.3.

Kod ovakvog pristupa vrijednosti rangova u trenutnoj iteraciji ovise samo od vrijednosti iz prethodne iteracije, ali ne od redoslijeda ili vrijednosti dobijenih u trenutnoj iteraciji. Posljedica je da se sve vrijednosti mogu računati paralelno, odnosno moguće je izvršiti paralelizaciju algoritma tako da svaki procesor obrađuje dio podataka. Međutim, izvjesno je da su vrijednosti u novoj iteraciji bliže konačnom rješenju, pa je pitanje možemo li iskoristiti te vrijednosti umjesto starih.

Posmatrajmo primjer sa slike 3.1



Slika 3.1: Primjer grafa za komparaciju

Predložena implementacija do rješenja stiže nakon 21 iteracije sa tolerancijom $\varepsilon = 1e{-}6$, a vrijednosti u pojedinim iteracijama prikazane su u tabeli 3.1.

Tabela 3.1: Iteracije pri standardnom pristupu

Iteracija / Rang	1	3	7	10	15	21
$\pi(A)$	0.090625	0.0967869	0.0958826	0.0957337	0.0957604	0.0957586
$\pi(B)$	0.267708	0.271268	0.273816	0.274227	0.274154	0.274158
$\pi(C)$	0.267708	0.271268	0.273816	0.274227	0.274154	0.274158
$\pi(D)$	0.373958	0.360678	0.356485	0.355812	0.355933	0.355925

Sada ćemo analizirati ponašanje algoritma kada koristimo najnovije poznate vrijednosti.
Početni rangovi (nulta iteracija):

$$\pi^{(0)}(A) = \frac{1}{4}, \quad \pi^{(0)}(B) = \frac{1}{4}, \quad \pi^{(0)}(C) = \frac{1}{4}, \quad \pi^{(0)}(D) = \frac{1}{4}$$

Prva iteracija:

$$\pi^{(1)}(A) = 0.15\frac{1}{4} + 0.85\frac{0.25}{4} = 0.090625$$

$$\pi^{(1)}(B) = 0.15\frac{1}{4} + 0.85(\frac{0.090625}{3} + \frac{0.25}{4} + \frac{0.25}{2}) = 0.222552$$
$$\pi^{(1)}(C) = 0.15\frac{1}{4} + 0.85(\frac{0.090625}{3} + \frac{0.25}{4} + \frac{0.25}{2}) = 0.222552$$
$$\pi^{(1)}(D) = 0.15\frac{1}{4} + 0.85(\frac{0.090625}{3} + \frac{0.222552}{1} + \frac{0.222552}{4}) = 0.299639$$

Uočimo da je zbir svih rangova sada 0.835368, stoga ćemo dobijeni rezultat normalizirati te nastaviti postupak, koji u ovom slučaju rezultat sa istom tolerancijom daje nakon 6 iteracija. Vrijednosti u pojedinim iteracijama prikazane su u tabeli 3.2.

Tabela 3.2: Iteracije pri modificiranom pristupu

Iteracija / Rang	1	2	3	4	5	6
$\pi(A)$	0.108485	0.0945838	0.0957655	0.0957623	0.0957584	0.0957586
$\pi(B)$	0.266412	0.274589	0.274166	0.274156	0.274158	0.274158
$\pi(C)$	0.266412	0.274589	0.274166	0.274156	0.274158	0.274158
$\pi(D)$	0.358691	0.356238	0.355902	0.355925	0.3559252	0.355925

Za toleranciju $\varepsilon = 1e{-}8$ prvi pristup terminira nakon 28 iteracija, a drugi nakon 7. Što se tiče implementacije, više nam nije potreban niz *previousPageRanks* obzirom da koristimo isključivo vrijednosti iz trenutne iteracije, a samim tim ni kopiranje niza *pageRanks* u *previousPageRanks*, čime opet postižemo memorijsku i vremensku uštedu. S druge strane, sada je doprinos apsorbirajućih čvorova neophodno računati za svaku stranicu ako želimo koristiti najnovije vrijednosti. Međutim, to je računski jako zahtjevno, pa je moguće napraviti kompromis da ipak na početku iteracije iskoristimo stare vrijednosti, a nauštrb povećanja broja iteracija, ali znatno manjeg u odnosu na prvi pristup. Programski kod modificirane metode nalazi se u prilogu.

3.1.1 Brzina konvergencije

Neka je matrica $\mathbf{H}'$ dimenzija $n \times n$ i njen spektar $(\lambda_1, \lambda_2, ..., \lambda_n)$. Kako je matrica $\mathbf{H}'$ nenegativna i stohastička po kolonama, na osnovu Perron-Frobenius teorema slijedi:

$$1 = \lambda_1 > |\lambda_2| \geq |\lambda_3| \geq |\lambda_4| \geq ... \geq |\lambda_n|. \tag{3.4}$$

Teorem 3.1.1 *Neka je* $\{1, \lambda_2, \lambda_3 ... \lambda_n\}$ *spektar matrice* $\mathbf{H}'$. *Spektar pozitivne stohastičke matrice* $\mathbf{G} = (1-d)\frac{1}{n}\mathbf{E} + d\mathbf{H}'$ *je tada* $\{1, d\lambda_2, d\lambda_3 ... d\lambda_n\}$.

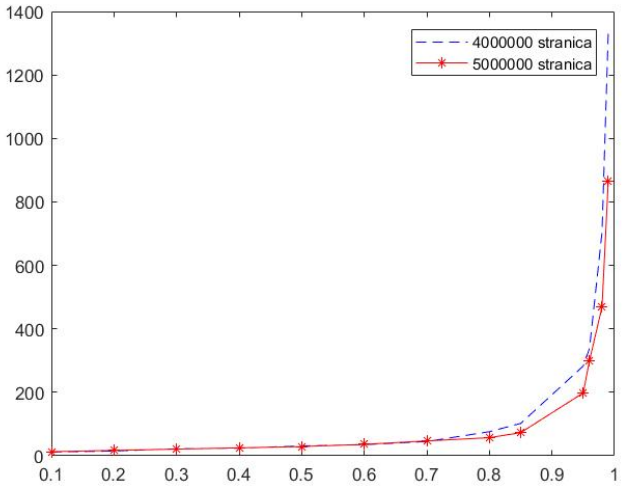
Dokaz se nalazi u [12]. Haveliwala i Kamvar u [13] pokazali su da je druga svojstvena vrijednost po veličini, reducibilne (kakav je ustvari Web graf) matrice $\mathbf{H}'$ zapravo faktor d, odnosno $\lambda_2(\mathbf{H}') = d$.

Brzina konvergencije metoda potencija je upravo uvjetovana drugom svojstvenom vrijednosti tj. odnosnom $\frac{\lambda_2}{\lambda_1}$. Kako je $\lambda_1 = 1$, konvergencija ovisi o $\lambda_2 = d$. Kako je nepromijenjeni graf Web-a reducibilan (obzirom da ne ispunjava uvjet jake povezanosti), možemo zaključiti da je brzina konvergencija određena brzinom kojom $d^k \to 0$, odnosno, što je manji faktor d,

algoritam brže konvergira. To i jeste u skladu sa intuicijom, obzirom da se tada u obzir manje uzima stvarna struktura i veze na grafu, a više faktor slučajnog skoka, koji je konstantan za sve čvorove, što znači i da se brže dostiže stabilnost.

Gruba procjena broja iteracija potrebnog da se dostigne tolerancija ε je $\frac{log_{10}\varepsilon}{log_{10}d}$. Za toleranciju $\varepsilon = 10^{-6}$ i $d = 0.85$ potrebno je oko $\frac{-6}{log_{10}0.85} \approx 85$ iteracija, a za $\varepsilon = 10^{-8}$ oko 110 iteracija. Naravno, radi se o gruboj procjeni, obzirom da struktura grafa značajno utječe na brzinu (a sve manje sa manjim d). Poređenja radi, za $d = 0.99$ potrebno je cca 1832 iteracija da se dostigne tolerancija $\varepsilon = 10^{-8}$. Naravno, ako ne skaliramo vrijednosti, procjena se mora uzeti relativno u odnosu na veličinu skupa, obzirom da bi za skup veličine $n = 1000000$ tolerancija od $\varepsilon = 10^{-5}$ bila zadovoljena već nakon jedne iteracije, obzirom da bi inicijalna distribucija bila reda 10^{-6}.

Na slici 3.2 nalazi se graf nacrtan u Matlabu, a prikazuje broj iteracija u ovisnosti od d za dva grafa veličine 4000000 i 5000000 čvorova i $\varepsilon = 1e{-}14$.



Slika 3.2: Ovisnost broja iteracija o koeficijentu d

Linearni sistem jednačina koji odgovara matrici $\mathbf{G}$ i vektoru π postaje loše uslovljen kada $d \to 1$, odnosno, male promjene u koeficijentima, dovode do velikih promjena u vektoru rješenja. Međutim, nama je od interesa ispitati osjetljivost iterativnog postupka na promjenu faktora d. Langville i Meyer su u [12] pokazali da je osjetljivost vektora π kao funkcije od d uvjetovana veličinom $\frac{1}{1-d}$. Izvjesno je da je neophodno napraviti kompromis: što je d bliže 1, osjetljivost raste, konvergencija usporava, a rangovi su nestabilniji. S druge strane, manji faktor d osigurava stabilnost, ali i nerealniji rezultat, obzirom da se umanjuje značaj strukture. Nasreću, najučestaliji izbor za $d = 0.85$, a za koji postupak prilično brzo konvergira, pri čemu ne zanemaruje strukturu grafa, posjeduje i intuitivno obrazloženje. Naime, u prosjeku 5 od 6 puta korisnik će za posjetu nekoj stranici slijediti hiperlink, a jednom će skočiti na neku nepovezanu stranicu.

3.2 Varijacije PageRank algoritma

3.2.1 Aitkenova ekstrapolacija

Aitkenova ekstrapolacija je jedan od metoda za ubrzanje kovergencije neke sekvence. Pretpostavimo da $x^{(k-2)}$ možemo predstaviti kao linearnu kombinaciju prva dva svojstvena vektora (odnosno vektora koja odgovaraju svojstvenim vrijednostima sa najvećim modulom). Kako se

radi o samo dvije svojstvene vrijednosti, jasno je da je to samo aproksimacija. Međutim, iz 3.4 možemo uočiti da utjecaj ostalih svojstvenih vrijednosti opada sa porastom k, tako da se shodno tome poboljšava i tačnost aproksimacije.

Aitkenova ekstrapolacija koristi 3 tačke, odnosno vrijednosti 3 iteracije [14]:

$$\mathbf{x}^{(\infty)} = \frac{\mathbf{x}^{(k-2)}\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\mathbf{x}^{(k-1)}}{\mathbf{x}^{(k)} - 2\mathbf{x}^{(k-1)} + \mathbf{x}^{(k-2)}}, \quad k = 2,3,4... \tag{3.5}$$

Sada ćemo ukratko obrazložiti rezon iza formule 3.5. Prvo ćemo istu dodatno transformisati, kako bismo je doveli u numerički stabilniji oblik:

$$\mathbf{x}^{(\infty)} = \frac{\mathbf{x}^{(k-2)}\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\mathbf{x}^{(k-1)}}{\mathbf{x}^{(k)} - 2\mathbf{x}^{(k-1)} + \mathbf{x}^{(k-2)}} = \tag{3.6}$$

$$= \frac{\mathbf{x}^{(k-2)}\mathbf{x}^{(k)} - 2\mathbf{x}^{(k-1)}\mathbf{x}^{(k-2)} + \mathbf{x}^{(k-2)}\mathbf{x}^{(k-2)} - \mathbf{x}^{(k-1)}\mathbf{x}^{(k-1)} + 2\mathbf{x}^{(k-1)}\mathbf{x}^{(k-2)} - \mathbf{x}^{(k-2)}\mathbf{x}^{(k-2)}}{\mathbf{x}^{(k)} - 2\mathbf{x}^{(k-1)} + \mathbf{x}^{(k-2)}}$$

$$= \frac{\mathbf{x}^{(k-2)}(\mathbf{x}^{(k)} - 2\mathbf{x}^{(k-1)} + \mathbf{x}^{(k-2)}) - (\mathbf{x}^{(k-1)}\mathbf{x}^{(k-1)} - 2\mathbf{x}^{(k-1)}\mathbf{x}^{(k-2)} + \mathbf{x}^{(k-2)}\mathbf{x}^{(k-2)})}{\mathbf{x}^{(k)} - 2\mathbf{x}^{(k-1)} + \mathbf{x}^{(k-2)}}$$

$$= \mathbf{x}^{(k-2)} - \frac{(\mathbf{x}^{(k-1)} - \mathbf{x}^{(k-2)})(\mathbf{x}^{(k-1)} - \mathbf{x}^{(k-2)})}{(\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}) - (\mathbf{x}^{(k-1)} - \mathbf{x}^{(k-2)})} \tag{3.7}$$

Kako bismo minimizirali greške pri zaokruživanju, brojnik i nazivnik smo predstavili u manje kompaktnom obliku, obzirom da u suprotnom postoji veća šansa gubljenja značajnih cifara.

Neka je sada $\mathbf{x}^{(k-2)}$ linearna kombinacija prva dva svojstvena vektora:

$$\mathbf{x}^{(k-2)} = \mathbf{v}_1 + \alpha_2\mathbf{v}_2 \tag{3.8}$$

U skladu sa rezonom iza jednačine 2.29, nakon uzastopnih množenja matricom $\mathbf{G}$ uzimajući u obzir da je $\lambda_1 = 1$ vrijedi:

$$\mathbf{x}^{(k-1)} = \mathbf{G}\mathbf{x}^{(k-2)} = \mathbf{v}_1 + \alpha_2\lambda_2\mathbf{v}_2 \tag{3.9}$$

$$\mathbf{x}^{(k)} = \mathbf{G}\mathbf{x}^{(k-1)} = \mathbf{v}_1 + \alpha_2\lambda_2^2\mathbf{v}_2 \tag{3.10}$$

Posmatrajmo sada brojnik i nazivnik umanjioca jednačine 3.5 za $i-ti$ element i označimo ih sa g_i i h_i respektivno:

$$g_i = (x_i^{(k-1)} - x_i^{(k-2)})^2 = ((v_1)_i + \alpha_2\lambda_2(v_2)_i - (v_1)_i - \alpha_2(v_2)_i)^2 = \alpha_2^2(\lambda_2 - 1)^2(v_2)_i^2 \tag{3.11}$$

$$h_i = x_i^{(k)} - 2x_i^{(k-1)} + x_i^{(k-2)} = (v_1)_i + \alpha_2\lambda_2^2(v_2)_i - 2(v_1)_i - 2\alpha_2\lambda_2(v_2)_i + (v_1)_i + \alpha_2(v_2)_i =$$

$$= \alpha_2(\lambda_2^2 - 2\lambda_2 + 1)(v_2)_i = \alpha_2(\lambda_2 - 1)^2(v_2)_i \tag{3.12}$$

Njihov količnik f_i iznosi:

$$f_i = \frac{\alpha_2^2(\lambda_2 - 1)^2(v_2)_i^2}{\alpha_2(\lambda_2 - 1)^2(v_2)_i} = \alpha_2(v_2)_i \tag{3.13}$$

Odnosno:

$$\mathbf{f} = \alpha_2 \mathbf{v}_2 \tag{3.14}$$

Otuda iz jednačine 3.8 slijedi rješenje za v_1 (odnosno vektor koji u našem slučaju predstavlja stacionarnu distribuciju π_i):

$$\mathbf{v}_1 = \mathbf{x}^{(k-2)} - \alpha_2 \mathbf{v}_2 = \mathbf{x}^{(k-2)} - \mathbf{f} = \mathbf{x}^{(k-2)} - \frac{\mathbf{g}}{\mathbf{h}} \tag{3.15}$$

Obzirom da je postupak zasnovan na pretpostavci da je $\mathbf{x}^{(k-2)}$ moguće predstaviti kao linearnu kombinaciju samo dva svojstvena vektora, jasno je da se radi samo o aproksimaciji $\mathbf{v_1}$. Međutim, ta aproksimacija se koristi da poboljša standardni metod potencija na način da se vrijednosti dobijene metodom potencija periodično ekstrapoliraju. Posljedica ekstrapolacije je da se vrijednosti stabiliziraju, te se tako konvergencija ubrzava.

Da bismo implementirali ovu funkciju, čiji se programski kod nalazi u prilogu, potreban nam je još jedan niz koji ćemo povremeno ažurirati i koji će služiti da pohrani vrijednosti $\mathbf{x}^{(k-2)}$, što je memorijska cijena ovog metoda. Što se tiče vremenske složenosti, ako koristimo vrijednosti izračunate u prethodne dvije iteracije, potrebno je još dodatno ažurirati vrijednosti po formuli 3.15, što je kompleksnosti $O(n)$ za skup stranica veličine n, a što je manje od ukupne kompleknosti jedne iteracije metoda potencija koja iznosi $O(m)$. Aitkenova ekstrapolacija se koristi za veće vrijednosti koeficijenta d tj. kada je d bliže 1, obzirom da tada metod potencija drastično sporije konvergira, te je potrebno povremeno "popraviti" rješenje. Ako se ne koristi koeficijent d približan 1, primjena ekstrapolacije ne donosi ubrzanja. Važan faktor je i učestalost primjene. Naime, ako se ekstrapolacija primjenjuje prečesto, proces ne konvergira. Uobičajeno je da se vrijednosti ekstrapoliraju svako 10 do 15 iteracija. Kamvar, Haveliwala, Manning i Golub u [15] navode da za $d = 0.99$ i $\varepsilon = 0.01$ primjena ekstrapolacije ubrzava proces za 38%, ali već za $\varepsilon = 0.002$, to ubrzanje iznosi svega 13%.

3.2.2 Kvadratna ekstrapolacija

Još jedan metod za ubrzanje konvergencije kada $d \to 1$ koji Kamvar, Haveliwala, Manning i Golub predlažu u [15] jeste *kvadratna ekstrapolacija*, a ovdje ćemo predstaviti dio koji se tiče QR faktorizacije (dekompozicije). Za razliku od Aitkenove ekstrapolacije, kvadratna ekstrapolacija podrazumijeva potprostor sačinjen od 3 svojstvena vektora. Međutim, pretpostavka od koje polazimo je da matrica $\mathbf{G}$ ima ukupno 3 svojstvena vektora, stoga je opet izvjesno da se radi o aproksimaciji.

Neka je $\mathbf{x}^{(k-3)}$ linearna kombinacija 3 svojstvena vektora:

$$\mathbf{x}^{(k-3)} = \mathbf{v}_1 + \alpha_2\mathbf{v}_2 + \alpha_3\mathbf{v}_3, \quad k = 3,4,5... \tag{3.16}$$

Obzirom da smo pretpostavili da matrica $\mathbf{G}$ ima samo 3 svojstvena vektora, karakteristični polinom matrice $\mathbf{G}$ možemo predstaviti kao:

$$P_G(\lambda) = \gamma_0 + \gamma_1\lambda + \gamma_2\lambda^2 + \gamma_3\lambda^3 \tag{3.17}$$

Kako je jedna (dominantna) svojstvena vrijednost matrice $\mathbf{G}$ $\lambda = 1$, a iz činjenice da su svojstvene vrijednosti korijeni svojstvenog polinoma, slijedi:

$$\gamma_0 + \gamma_1 + \gamma_2 + \gamma_3 = 0 \tag{3.18}$$

Cayley-Hamiltonov stav navodi da je svaka matrica nula operator svog karakterističnog polinoma, odnosno svaka matrica zadovoljava svoj karakteristični polinom. Otuda slijedi da je $P_G(\mathbf{G}) = \mathbf{0}$, odnosno, $\forall\mathbf{v}$ u $\mathbb{R}$:

$$P_G(\mathbf{G}) = \mathbf{0} \implies [\gamma_0\mathbf{I} + \gamma_1\mathbf{G} + \gamma_2\mathbf{G}^2 + \gamma_3\mathbf{G}^3]\mathbf{v} = \mathbf{0} \tag{3.19}$$

Neka je $\mathbf{v} = \mathbf{x}^{(k-3)}$. Tada iz $\mathbf{x}^{(t+2)} = \mathbf{G}\mathbf{x}^{(t+1)} = \mathbf{G}^2\mathbf{x}^{(t)}$, $t = 0,1,2...$ slijedi:

$$[\gamma_0\mathbf{I} + \gamma_1\mathbf{G} + \gamma_2\mathbf{G}^2 + \gamma_3\mathbf{G}^3]\mathbf{x}^{(k-3)} = \mathbf{0} \tag{3.20}$$

$$\gamma_0\mathbf{x}^{(k-3)} + \gamma_1\mathbf{G}\mathbf{x}^{(k-3)} + \gamma_2\mathbf{G}^2\mathbf{x}^{(k-3)} + \gamma_3\mathbf{G}^3\mathbf{x}^{(k-3)} = \mathbf{0} \tag{3.21}$$

$$\gamma_0\mathbf{x}^{(k-3)} + \gamma_1\mathbf{x}^{(k-2)} + \gamma_2\mathbf{x}^{(k-1)} + \gamma_3\mathbf{x}^{(k)} = \mathbf{0} \tag{3.22}$$

Iz 3.18 slijedi:

$$(-\gamma_1 - \gamma_2 - \gamma_3)\mathbf{x}^{(k-3)} + \gamma_1\mathbf{x}^{(k-2)} + \gamma_2\mathbf{x}^{(k-1)} + \gamma_3\mathbf{x}^{(k)} = \mathbf{0} \tag{3.23}$$

Pregrupisanjem dobijamo:

$$(\mathbf{x}^{(k-2)} - \mathbf{x}^{(k-3)})\gamma_1 + (\mathbf{x}^{(k-1)} - \mathbf{x}^{(k-3)})\gamma_2 + (\mathbf{x}^{(k)} - \mathbf{x}^{(k-3)})\gamma_3 = \mathbf{0} \tag{3.24}$$

Radi kompaktnijeg zapisa, uvest ćemo sljedeće smjene:

$$\mathbf{y}^{(k-2)} = \mathbf{x}^{(k-3)} - \mathbf{x}^{(k-2)} \tag{3.25}$$

$$\mathbf{y}^{(k-1)} = \mathbf{x}^{(k-1)} - \mathbf{x}^{(k-3)} \tag{3.26}$$

$$\mathbf{y}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^{(k-3)} \tag{3.27}$$

Sada jednačinu 3.24 možemo napisati u matričnom obliku kao:

$$\begin{pmatrix}\mathbf{y}^{(k-2)} & \mathbf{y}^{(k-1)} & \mathbf{y}^{(k)}\end{pmatrix}\gamma = 0 \tag{3.28}$$

Dakle, potrebno je jednačinu 3.28 riješiti po γ. Kako nam nije od interesa trivijalno rješenje, ograničit ćemo jedan koeficijent na $\gamma_3 = 1$, obzirom da to neće utjecati na korijene polinoma. Jednačina 3.28 postaje:

$$\begin{pmatrix}\mathbf{y}^{(k-2)} & \mathbf{y}^{(k-1)}\end{pmatrix}\begin{pmatrix}\gamma_1 \\ \gamma_2\end{pmatrix} = -\mathbf{y}^{(k)} \tag{3.29}$$

Obzirom da su dvije nepoznate veličine, a broj jednačine je jednak broju čvorova n ($n > 2$), radi se o preodređenom sistemu jednačina. Kako takvi sistemi često nemaju rješenje, potrebno je pronaći rješenje koje pribiližno zadovoljava sve jednačine, a najjednostavniji problem za rješavanje je *rješavanje u smislu najmanjih kvadrata*. Rješavanje preodređenih sistema u smislu najmanjih kvadrata može se svesti na rješavanje normalnih sistema [16], međutim takvi sistemi su obično vrlo loše uslovljeni, te se ta mogućnost ne upotrebljava. Bolji pristup je *QR dekompozicija (faktorizacija)*.

Dakle, potrebno je izvršiti faktorizaciju matrice $\begin{pmatrix}\mathbf{y}^{(k-2)} & \mathbf{y}^{(k-1)}\end{pmatrix}$ = **QR**, zatim izvršiti jedno množenje slijeva, te riješiti trougaoni sistem supstitucijom unazad.

Najjednostavniji algoritam za pronalaženje QR faktorizacije je **Gram-Schmidtov algoritam**, međutim glavni nedostatak je numerička nestabilnost. Najjednostavniji među numerički vrlo stabilnim algoritmima je **Householderov algoritam**. Kompleksniji, ali pogodniji za matrice koje imaju mnogo nultih elemenata (engl. sparse matrices), obzirom da se ne mora pamtiti čitava matrica, niti vršiti operacije nad nultim elementima je **Givensov metod**. Dodatna prednost jeste, što je za razliku od Householderovog, ovaj metod pogodan za paralelizaciju, ali ipak predstavlja implementacijski izazov, te je opet potrebno pronaći kompromis između jednostavnosti i efikasnosti.

Primjenom Cayley-Hamiltonovog stava na polinom $\frac{P_G(\lambda)}{\lambda-1}$, i transformacija kao u uvodnom dijelu (detaljnije u [15]), dolazimo do konačne formule za svojstveni vektor $\mathbf{v_1}$, odnosno vektor ka kome konvergira iterativni proces:

$$\mathbf{v_1} = \beta_0\mathbf{x}^{(k-2)} + \beta_1\mathbf{x}^{(k-1)} + \beta_2\mathbf{x}^{(k-3)} \tag{3.30}$$

gdje je:

$$\beta_0 = \gamma_1 + \gamma_2 + \gamma_3 \tag{3.31}$$

$$\beta_1 = \gamma_2 + \gamma_3 \tag{3.32}$$

$$\beta_3 = \gamma_3 \tag{3.33}$$

Iz tabele 3.3 možemo uočiti da kvadratna ekstrapolacija postiže značajno ubrzanje kada $d \to 1$, obzirom da nad velikim grafovima (kao što je web graf), računanje može trajati danima, te s tim u vezi, svaka ušteda znači. Cijena koja se plaća za to ubrzanje je računanje QR faktorizacije, kao i memorijsko opterećenje (dodatni vektori za pamćenje vrijednosti prethodnih iteracija). Međutim, kao i Aitkenova ekstrapolacija, kvadratna ekstrapolacija se primjenjuje periodično i ne prečesto (cca svako 15 iteracija), tako da je moguće vrijednosti prethodnih operacija pamtiti

Tabela 3.3: Vremenska ušteda primjenom kvadratne ekstrapolacije

d	ε	*vremenska ušteda*
0.9	0.001	23%
0.95	0.001	31%
0.99	0.018	69%

na disku. Kvadratna ekstrapolacija brže konvergira od Aitkenove ekstrapolacije, a što je u skladu sa intuicijom, obzirom da je aproksimacija pomoću tri svojstvena vektora, bliža stvarnoj vrijednosti, nego ona sa dva. Osim toga, kvadratna ekstrapolacija ne generiše šiljke koji se javljaju kod Aitkenove ekstrapolacije zbog loše aproksimacije drugog svojstvenog vektora, a zbog čega prečesta primjena Aitkenove ekstrapolacije dovodi do divergencije procesa.

3.2.3 Personalizirani PageRank algoritam

Personalizirani PageRank predstavlja kontekstno, tematski ili korisnički orijentisan PageRank. Kontekstno neovisno rangiranje je isto za sve korisnike, odnosno bilo kakve dodatne informacije o onom ko pretražuje i tematici od interesa ne igraju ulogu u krajnjem rezultatu. S druge strane, korisnik koji se trenutno nalazi u Sarajevu, na upit o najboljim restoranima očekuje restorane koji su u Sarajevu ili blizini, za razliku od korisnika u Parizu. U tom slučaju ključni parametar je lokacija. Analogno, pojam "atribut" ima različitu konotaciju u kontekstu lingvistike i objektno-orijentisanog programiranja, te bi takav upit trebao dati prednost onim stranicama koje su prilagođene traženoj tematici.

Osnovna ideja je da vjerovatnoće slučajnog skoka nisu uniformno raspoređene, nego u skladu sa preferencijama, odnosno, vektor **e**, mijenjamo personaliziranim (teleportacijskim) vektorom **u**:

$$\pi_p = (1-d)\frac{1}{n}\mathbf{u} + d\mathbf{H}'\pi_p \tag{3.34}$$

Dakle, skok se dešava samo na unaprijed određeni podskup stranica. Međutim, ovom modifikacijom, nemamo garanciju da je graf jako povezan, a što je preduslov egzistencije i konvergencije rješenja. Obzirom da se sada radi o lokalnom rangiranju (vezanom za neku temu), stranice koje ni na koji način nisu povezane sa našim startnim skupom stranica (startni skup čine stranice koje odgovaraju nenegativnim elementima personaliziranog vektora), ni ne moraju biti uzete u razmatranje, odnosno sasvim je legitimno posmatrati samo one stranice koje pripadaju komponentama povezanosti induciranim startnim skupom stranica.

Danas, personalizirani PageRank ima široku primjenu u sistemima za preporuku, marketingu i detekciji vještački konstruisanih linkova sa ciljem povećanja ranga (engl. spam farms, link farms). Na primjer, Twitter koristi personalizirani PageRank da kreira sugestije korisnicima o tome koga bi narednog željeli zapratiti [17].

Interesantno je da je personalizirani PageRank primjenu našao i u biohemiji. Naime, upotreba antibiotika uglavnom dovodi do mutacije organizma patogena te otpornosti na lijek, čime se javlja potreba za razvojem novog lijeka. Da bi se razvio novi lijek, potrebno je pronaći metu u uzročniku, odnosno mikroorganizmu patogena, za koji se vežu molekularni spojevi kreirani da iste inhibiraju. Biohemijske reakcije tretiraju se kao čvorovi metaboličke mreže, a smatra se da bi za analizu takve mreže i pronalaženje meta odnosno ciljnih proteina, mogao bolje poslužiti

PageRank nego metoda koja u obzir uzima samo broj ulaznih čvorova [18] [19].

3.2.4 BlockRank algoritam

Makroskopska struktura Web-a sugeriše da se Web sastoji od različitih komponenti povezanosti. Međutim, interno se svaka od tih komponenti sastoji od blokova, odnosno skupina Web stranica, koje imaju interakciju međusobno (mogu biti na istom hostu ili domeni), a manje sa ostalim blokovima. Ako blokove (hostove) posmatramo kao čvorove u grafu, dobijamo kompresovani Web graf (Host graf). Kamwar i drugi [20] ovakvu topologiju koriste za ubrzanje PageRank algoritma.

Obzirom da svaki blok ima mali broj ulaznih i izlaznih linkova u komparaciji sa internim, za svaki blok računaju se vrijednosti ranga stranica na osnovu internih linkova standardnim PageRank algoritmom. Nakon toga, računa se relativni značaj svakog bloka (BlockRank) na osnovu izračunatih lokalnih rangova i stvarnih distribucija na Web grafu, odnosno veza pojedinih blokova. Izračunati lokalni rangovi stranica se zatim množe sa relativnim značajem bloka u kome se stranica nalazi, čime se dobija globalni rang, odnosno aproksimacija stvarnih vrijednosti rangova. Konačno, te vrijednosti se koriste kao ulaz standardnom algoritmu, obzirom da je aproksimacija bliža rješenju od uniformne distribucije.

Ovakav pristup ima nekoliko prednosti:

- Lokalni rangovi mnogih blokova konvergiraju brzo, tako da algoritam za te blokove terminira nakon nekoliko iteracija. S druge strane, kod standardnog pristupa, sporokonvergirajući "blokovi" predstavljaju usko grlo, obzirom da se zbog njih mnogo resursa troši na ponovno računanje vrijednosti blokova čije su rangovi već konvergirali.
- Računanje lokalnih rangova može se obaviti distribuirano ili paralelno odmah nakon prikupljanja informacija o tom bloku neovisno o ostalim blokovima
- Izračunate vrijednosti lokalnih rangovi mogu se iskoristiti i u budućnosti prilikom ažuriranja. Recimo da se pojedini blokovi mijenjaju frekventnije od drugih. U tom slučaju, za računanje globalnih rangova potrebno je ponovo izračunati lokalne vrijednosti rangova samo za te blokove, a za ostale blokove iskoristiti poznate vrijednosti

Obzirom da računanje personaliziranog PageRanka za razne vrijednosti teleportacijskog vektora nije efikasno, postoji varijanta BlockRank algoritma i za tu svrhu. Zasniva se na ideji da teleportacijski vektor sada sadrži vjerovatnoće skoka na određene blokove, odnosno računanje se vrši nad kompresovanim Host grafom. Na taj način, lokalni rangovi su isti za sve personalizacije, a razlikuje se drugi korak, odnosno računanje BlockRanka u ovisnosti od vektora teleportacije, čime se postiže da je jednostavnije, odnosno "jeftinije" računati personalizirane rangove za razne vrijednosti teleportacijskog vektora.

3.2.5 Linearni stacionarni iterativni metodi

Kao što smo spomenuli, jedan od načina rješavanja PageRank problema je i rješavanje sistema linearnih jednačina, pri čemu postoji više formulacija tog sistema [21]. Obzirom da smo naglasili da direktno rješavanje nije upotrebljivo za problem ove veličine, u nastavku ćemo se ukratko osvrnuti na metode za iterativno rješavanje linearnih sistema jednačina, te u kojoj su oni vezi sa već predloženim metodom potencija.

Neka je zadan linearni sistem jednačina $\mathbf{Ax} = \mathbf{b}$, pri čemu je matrica $\mathbf{A}$ dimenzija $n \times n$, $det(\mathbf{A}) \neq 0$, a $\mathbf{b}$ i $\mathbf{x}$ dimenzija $n \times 1$.
Matricu $\mathbf{A}$ ćemo napisati u obliku razlike dvije matrice: $\mathbf{M}$ i $\mathbf{N}$:

$$\mathbf{A} = \mathbf{M} - \mathbf{N}, \tag{3.35}$$

gdje je $\mathbf{M}$ invertibilna matrica.

Iz 3.35 slijedi:

$$(\mathbf{M} - \mathbf{N})\mathbf{x} = \mathbf{b} \tag{3.36}$$

$$\mathbf{Mx} = \mathbf{Nx} + \mathbf{b} \tag{3.37}$$

Nakon lijevog množenja sa $\mathbf{M}^{-1}$ dobijamo:

$$\mathbf{x} = \mathbf{M}^{-1}\mathbf{Nx} + \mathbf{M}^{-1}\mathbf{b} \tag{3.38}$$

Konačno dolazimo do iterativnog postupka:

$$\mathbf{x}^{(k+1)} = \mathbf{M}^{-1}\mathbf{Nx}^{(k)} + \mathbf{M}^{-1}\mathbf{b} \tag{3.39}$$

Neka je vektor $\mathbf{d} = \mathbf{M}^{-1}\mathbf{b}$, i matrica $\mathbf{H} = \mathbf{M}^{-1}\mathbf{N}$, koju ćemo nazivati *iterativna matrica*.
Za proizvoljni inicijalni vektor $\mathbf{x}^{(0)}$ ($\mathbf{x}^{(0)} \in \mathbb{C}^n$), sekvenca definisana kao:

$$\mathbf{x}^{(k+1)} = \mathbf{Hx}^{(k+1)} + \mathbf{d}, \quad k = 0, 1, 2... \tag{3.40}$$

naziva se *linearna stacionarna iteracija*.
Potreban i dovoljan uslov da iterativni postupak 3.40 konvergira je da je dominantna svojstvena vrijednost matrice (spektralni radijus) $\mathbf{H}$ po modulu < 1.

Neka je sada matrica $\mathbf{A}$ jednaka:

$$\mathbf{A} = \mathbf{D} - \mathbf{L} - \mathbf{U}, \tag{3.41}$$

gdje $\mathbf{D}$ predstavlja dijagonalni dio, $-\mathbf{L}$ donji trougaoni, a $-\mathbf{U}$ gornji trougaoni dio matrice $\mathbf{A}$. U ovisnosti od toga koji dio ($\mathbf{D}$, $\mathbf{L}$ ili $\mathbf{U}$) pripada kojem članu razlike ($\mathbf{M}$ ili $\mathbf{N}$), postoje različiti iterativni algoritmi. Tri najpoznatija su: Jacobijev metod, Gauss-Seidelov metod i SOR (engl. Successive Overrelaxation) metod [22], a načini rastavljanja matrice $\mathbf{A}$ sva tri metoda i iterativne sheme prikazani su u tabelama 3.4 i 3.5.

Može se pokazati da je primjena Jacobijevog postupka na sistem jednačina koji odgovara PageRank problemu zapravo ekvivalentan primjeni *metoda potencija* [21] za grafove bez apsorbirajućih čvorova (a to svakako postižemo predloženim modifikacijama). Gauss-Seidelov metod je sličan Jacobijevom, osim što koristi najnovije poznate vrijednosti i samim tim brže konvergira od Jacobijevog, a upravo od redoslijeda računanja ovisi brzina konvergencije. Konačno, SOR metod uvodi i dodatni parametar relaksacije ω, a pogodnim izborom tog parametra, ovaj metod konvergira brže od Gauss-Seidelovog.

Tabela 3.4: Rastavljanje matrice za Jacobi, Gauss-Seidel i SOR metod

Metod	**M**	**N**
Jacobi	$\mathbf{D}$	$\mathbf{L}+\mathbf{U}$
Gauss-Seidel	$\mathbf{D}-\mathbf{L}$	$\mathbf{U}$
SOR	$\omega^{-1}(\mathbf{D}-\omega\mathbf{L})$	$(\omega^{-1}-1)\mathbf{D}+\mathbf{U}$

Tabela 3.5: Iterativna shema Jacobi, Gauss-Seidel i SOR metoda

Metod	Iterativni postupak
Jacobi	$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}\mathbf{L}\mathbf{x}^{(k)} + \mathbf{D}^{-1}\mathbf{U}\mathbf{x}^{(k)} + \mathbf{D}^{-1}\mathbf{b}$
Gauss-Seidel	$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}\mathbf{L}\mathbf{x}^{(k+1)} + \mathbf{D}^{-1}\mathbf{U}\mathbf{x}^{(k)} + \mathbf{D}^{-1}\mathbf{b}$
SOR	$\mathbf{x}^{(k+1)} = (1-\omega)\mathbf{x}^{(k)} + \omega\mathbf{D}^{-1}(\mathbf{b} - \mathbf{L}\mathbf{x}^{(k)} - \mathbf{U}\mathbf{x}^{(k)})$

Postoji još niz iterativnih metoda koji se mogu primijeniti na PageRank problem [23], a predstavili smo ova tri kako bismo pokazali da se postojeći postupci i algoritmi pogodnim transformacijama mogu primijeniti na pronalazak ispravnog rangiranja stranica.

* * *

U ovom poglavlju predložena je implementacija PageRank algoritma u programskom jeziku C++ kroz iterativni metod potencija, a uz upotrebu jednostruko povezane liste kao strukture podataka s ciljem postizanje memorijske uštede. U tu svrhu, uveli smo novu formulu iterativnog procesa prilagođenu pamćenju isključivo neophodnog dijela Google matrice, te predložili jednu modifikaciju čija je posljedica smanjenje broja iteracija. Nadalje, pokazana je ovisnost brzine konvergencije o faktoru prigušenja d, te su obuhvaćeni metodi za ubrzanje kada je ta konvergencija spora, i modifikacije koje omogućavaju rangiranje u okviru konteksta ili aproksimaciju rješenja.

Zaključak

PageRank je algoritam za globalno rangiranje stranica zasnovan na korištenju informacija sadržanih u strukturi Web grafa, na način da svaka stranica svoj značaj propagira kroz strukturu grafa, što tu definiciju čini rekurzivnom, a rang relativnim. Uspješni i efikasni algoritmi pretežno su bazirani na opsežnoj matematičkoj teoriji. Analiza egzistencije, jedinstvenosti i konvergencije rješenja PageRank problema te svojstva i neophodne korekcije koje to garantuju temeljeni su na sintezi i dostignućima teorije grafova, vjerovatnoće i linearne algebre. Posljedica tih analiza i zapažanja je uvođenja elementa slučajnosti, odnosno slučajnog skoka na bilo koji čvor, a koji je zapravo i u skladu sa intuitivnom predstavom o ponašanju na Web-u. U osnovi, PageRank svodi se na dva poznata problema: pronalazak svojstvenog vektora koji odgovara dominantoj svojstvenoj vrijednosti i rješavanje homogenog sistema jednačina. Kod pristupa sa svojstvenim vrijednostima, najpoznatiji metod je metod potencija. S druge strane, za rješavanje sistema jednačina koriste se Jacobi, Gauss-Seidel i SOR metod. Periodičnom primjenom nekog metoda za akceleraciju, kao što su Aitkenova ili Kvadratna ekstrapolacija, može se smanjiti broj iteracija.

Iako se implementacija algoritma doima izravnom i jednostavnom, za problem ove dimenzionalnosti ipak zahtijeva kreativnost i domišljatost kako bi se dostupni resursi racionalno iskoristili, a izračunavanje izvršilo u što kraćem vremenskom roku. Optimizacija memorijskog zauzeća i efikasniji numerički metodi područja su sadašnjih i budućih istraživanja, a fokus je na prikladnom izboru strukture podataka, paralelizaciji i distribuciji izračunavanja, kompresiji problema, aproksimaciji početne distribucije te problemu ažuriranja grafa i ponovnoj upotrebi prethodno dobijenih rezultata.

Ostvareni ciljevi završnog rada

PageRank je i danas u upotrebi u svojoj osnovnoj namjeni, ali je zbog veličine i dinamike Web-a, korisničkih i očekivanja tržišta, te napretka u prikupljanju i obradi podataka, problem kompleksniji, ali time i interesantniji za nove interpretacije i perspektive posmatranja.

Pokazali smo da je odlika ovog problema mogućnost inkorporiranja već poznatih i razrađenih metoda, a tome svjedoče i primjena QR faktorizacije, Monte Carlo metoda [24], paralelizacija bazirana na sistemima linearnih jednačina [25] i particionisanju hipergrafa [26] te upotreba grafičke procesorske jedinice za ubrzanje [27].

Motivacija za takav interes je što je PageRank zapravo univerzalan koncept: bilo koji problem koji se može modelirati kao graf koji predstavlja veze i tokove među čvorovima, a gdje nam je od interesa mjera interakcije ili međusobnog utjecaja čvorova, odnosno određivanje značaja svakog čvora, zapravo možemo mapirati na PageRank. Tako su danas razvijeni algoritmi u biologiji i bioinformatici bazirani na PageRanku (GeneRank [28] i ProteinRank [29]), zatim algoritmi za predviđanje gužvi na putevima i kretanja ljudi [30], a intenzivno se koristi i u sportu za rangiranje timova, u bibliometriji te ekonomskim, kompjuterskim i socijalnim mre-

žama. Ovo je još jedan dokaz o cjelovitosti nauke te univerzalnosti naučnih otkrića obzirom da se dostignuća u okviru jedne, koriste i podstiču istraživanja u drugim naukama, sa posebnim akcentom na neraskidivu vezu kompjuterskih nauka i matematike. Diverzitet primjene i mogućnost kombiniranja sa postojećim metodima, daju prostor i mogućnost za širenje znanja i tehnološko unapređivanje.

Prilozi

Program 4: Programski kod metode za generisanje datoteke sa strukturom grafa

```
void DatasetGenerator::generateDataset(int size, const std::string&
    outputFile, int maxOuterLinks) {
    std::ofstream outputStream(outputFile);
    for(int i = 0; i < size; i++){
        int numOfOuterLinks = rand()%maxOuterLinks;
        PageGenerator pageGenerator(numOfOuterLinks, 0, size - 1);
        outputStream << i <<": ";
        while(pageGenerator.has_next()){
            outputStream << pageGenerator.next();
            if (pageGenerator.has_next()) outputStream << ", ";
        }
        outputStream << std::endl;
    }
    outputStream.close();
}
```

Program 5: Programski kod metode za učitavanje grafa iz fajla

```
void PageRank::loadData(const std::string &fileName) {
    std::ifstream page_data(fileName);
    if(!page_data) {
        std::cout<<"File opening failed"<<std::endl;
        return;
    }

    int  page_number, danglingNodesCounter = 0;
    char znak;
    std::string outer_links_str;
    countPagesAndDanglingNodes(fileName);
    initialize();

    while(page_data>>page_number>>znak && std::getline(page_data,
        outer_links_str)) {
        std::stringstream ss(outer_links_str);
        int counter(countPages(outer_links_str));

        for (int i; ss >> i;) {
            if (ss.peek() == ',')
                ss.ignore();
            addLinkToPage(page_number, i);
        }
        if(!counter)
            danglingNodes[danglingNodesCounter++]  = page_number;
        else
            outerLinkDistributions[page_number] = 1.f / counter;
    }
    page_data.close();
}
```

Program 6: Programski kod metode za računanje doprinosa apsorbirajućih čvorova

```
double PageRank::getDanglingNodesContribution() {
    double sum = 0;
    for(int i = 0; i < numberOfDanglingNodes; i++){
        sum += pageRanks[danglingNodes[i]];
    }
    return sum/numberOfPages;
```

```
}
```

Program 7: Programski kod metode za računanje doprinosa ulaznih linkova

```
double LinkedList::multiplyDistributionWithBacklinks(const float*
    probDistribution, const float* pageRanks) {
    double sum = 0;
    Node* tmp = head;
    while(tmp){
        sum += (probDistribution[tmp->link])*pageRanks[tmp -> link];
        tmp = tmp->next;
    }
    return sum;
}
```

Program 8: Programski kod metode za računanje PageRank-a

```
void PageRank::calculateSimplePageRank() {
    float* previousPageRanks = new float[numberOfPages]{};
    double randomWalkProb = (1 - dampingFactor)/numberOfPages, maxError =
        1;

    while(maxError > eps && iterations < maxNumberOfIterations){
        double danglingNodesAndRandomWalk =  dampingFactor*
            getDanglingNodesContribution() + randomWalkProb;
        std::copy(pageRanks, pageRanks + numberOfPages, previousPageRanks
            );
        maxError = 0;
        for(int i = 0; i < numberOfPages; i++){
            pageRanks[i] = dampingFactor * pLinkedLists[i].
                multiplyDistributionWithBacklinks(outerLinkDistributions,
                previousPageRanks) + danglingNodesAndRandomWalk;

            double pageError = abs(previousPageRanks[i] - pageRanks[i]);
            if(pageError > maxError ) maxError = pageError;
        }
        iterations++;
    }
    delete[] previousPageRanks;
}
```

Program 9: Programski kod modificirane metode za računanje PageRank-a

```
void PageRank::calculateSimplePageRankModified(){

    double randomWalkProb = (1 - dampingFactor)/numberOfPages, maxError =
        1, prevRank = 0;
    int iterations = 0, indexOfMaxPageErr = 0;
    while(maxError > eps && iterations < maxNumberOfIterations){
        maxError = 0;
        for(int i = 0; i < numberOfPages; i++){
            double tmp = pageRanks[i];
            pageRanks[i] = dampingFactor *
                    (pLinkedLists[i].multiplyDistributionWithBacklinks(
                        outerLinkDistributions, pageRanks) +
                        getDanglingNodesContribution()) + randomWalkProb;

            double pageError = abs(pageRanks[i] - tmp);
```

```
13              if(pageError > maxError) {
14                  maxError = pageError;
15                  indexOfMaxPageErr = i;
16                  prevRank = tmp;
17              }
18          }
19          double sum = 0;
20          sum = std::accumulate(pageRanks, pageRanks + numberOfPages, sum);
21          std::transform(pageRanks, pageRanks + numberOfPages, pageRanks, [
                sum](float x){
22              return x/sum;
23          });
24          maxError = abs(pageRanks[indexOfMaxPageErr] - prevRank);
25          iterations++;
26      }
27  }
```

Program 10: Programski kod metode za upisivanje rezultata u datoteku

```
1  void PageRank::writePageRanksToFile(const std::string& outputFileName) {
2      std::ofstream outputStream(outputFileName);
3      for(int i = 0; i < numberOfPages; i++){
4          outputStream<< std::setprecision(13) << "p("<<i<<"): "<<pageRanks
               [i]<<std::endl;
5      }
6      outputStream.close();
7  }
```

Program 11: Programski kod funkcije za računanje Aitkenove ekstrapolacije

```
1  void performAitkensExtrapolation(float* x_2, float* x_1, float* x, int
       size){
2      for(int i = 0; i < size; i++){
3          double g_i = (x_1[i] - x_2[i])*(x_1[i] - x_2[i]);
4          double h_i = (x[i] - x_1[i]) - (x_1[i] - x_2[i]);
5          x[i] -= g_i/h_i;
6      }
7  }
```

Literatura

[1] Brin, S., Lawrence Page, F., Raghavan, P., Rajagopalan, S., Stata, R., Tomkins, A., Wiener, J., "The anatomy of a large-scale hypertextual web search engine".

[2] Tanase, R., Radu, R., "Lecture# 3: Pagerank algorithm–the mathematics of google search", Online: http://www. math. cornell. edu/mec/Winter2009/RalucaRemus/Lecture3/lecture3. html, 2016.

[3] Black, P. E., "Dictionary of algorithms and data structures", 1998, dostupno na: https://www.nist.gov/dads/HTML/graph.html

[4] Željko Jurić, "Elementi teorije grafova", dostupno na: https://c2.etf.unsa.ba/pluginfile.php/50054/mod_resource/content/3/Predavanje%209.pdf 2019.

[5] Peretti, A., Roveda, A., "On the mathematical background of google pagerank algorithm", 2014.

[6] Željko Jurić, "Uvod u diskretnu teoriju vjerovatnoće", dostupno na: https://c2.etf.unsa.ba/pluginfile.php/49396/mod_resource/content/3/Predavanje%205.pdf 2019.

[7] Huzak, M., "Vjerojatnost i matematicka statistika", Predavanja (skripta), dostupno na http://aktuari. math. pmf. unizg. hr/docs/vms. pdf, 2006.

[8] Fewster, R., "Chapter 8: Markov chains", Predavanje, dostupno na https://www.stat.auckland.ac.nz/ fewster/325/notes/ch8.pdf.

[9] Freedman, A., "Convergence theorem for finite markov chains", Proc. REU, 2017.

[10] Broder, A., Kumar, R., Maghoul, F., Raghavan, P., Rajagopalan, S., Stata, R., Tomkins, A., Wiener, J., "Graph structure in the web", Computer networks, Vol. 33, No. 1-6, 2000, str. 309–320.

[11] Šupić, H., "Analiza vremenske složenosti algoritama", dostupno na: https://c2.etf.unsa.ba/pluginfile.php/43610/mod_resource/content/1/2014/Predavanje02.pdf

[12] Amy N. Langville, C. D. M., "Deeper inside pagerank", Predavanje, dostupno na https://galton.uchicago.edu/ lekheng/meetings/mathofranking/ref/langville.pdf.

[13] Haveliwala, T. H., Kamvar, S. D., "The second eigenvalue of the google matrix", 2003.

[14] Tiruneh, A. T., "Higher order aitken extrapolation with application to converging and diverging gauss-seidel iterations", arXiv preprint arXiv:1310.4288, 2013.

[15] Kamvar, S. D., Haveliwala, T. H., Manning, C. D., Golub, G. H., "Extrapolation methods for accelerating pagerank computations", in Proceedings of the 12th international conference on World Wide Web, 2003, str. 261–270.

[16] Željko Jurić, "Preodređeni sistemi jednačina", str. 8–18, dostupno na: https://c2.etf.unsa.ba/pluginfile.php/49535/mod_resource/content/3/Predavanje%205.pdf 2019.

[17] Gupta, P., Goel, A., Lin, J., Sharma, A., Wang, D., Zadeh, R., "Wtf: The who to follow service at twitter", in Proceedings of the 22nd International Conference on World Wide Web, ser. WWW '13. New York, NY, USA: Association for Computing Machinery, 2013, str. 505–514, dostupno na: https://doi.org/10.1145/2488388.2488433

[18] Iván, G., Grolmusz, V., "When the Web meets the cell: using personalized PageRank for analyzing protein interaction networks", Bioinformatics, Vol. 27, No. 3, 12 2010, str. 405-407, dostupno na: https://doi.org/10.1093/bioinformatics/btq680

[19] Bánky D, G. V., Iván G, "Equal Opportunity for Low-Degree Network Nodes: A PageRank-Based Method for Protein Target Identification in Metabolic Graphs", dostupno na: https://doi.org/10.1371/journal.pone.0054204

[20] Kamvar, S., Haveliwala, T., Manning, C., Golub, G., "Exploiting the block structure of the web for computing pagerank", Stanford, Tech. Rep., 2003.

[21] Silvestre, D., Hespanha, J., Silvestre, C., "A pagerank algorithm based on asynchronous gauss-seidel iterations", in 2018 Annual American Control Conference (ACC). IEEE, 2018, str. 484–489.

[22] A. Arasu, A. T., J. Novak, Tomlin, J., "PageRank computation and the structure of the web: Experiments and algorithms. In Proceedings of the Eleventh International World Wide Web Conference", 2002, str. 107-117, dostupno na: https://doi.org/10.1371/journal.pone.0054204

[23] Rodrigues, I. C. d. S. B., Pinto, M., "Pagerank: how to accelerate its computation", 2018.

[24] Avrachenkov, K., Litvak, N., Nemirovsky, D., Osipova, N., "Monte carlo methods in pagerank computation: When one iteration is sufficient", SIAM Journal on Numerical Analysis, Vol. 45, No. 2, 2007, str. 890–904.

[25] Gleich, D., Zhukov, L., Berkhin, P., "Fast parallel pagerank: A linear system approach", Yahoo! Research Technical Report YRL-2004-038, available via http://research. yahoo. com/publication/YRL-2004-038. pdf, Vol. 13, 2004, str. 22.

[26] Bradley, J. T., de Jager, D. V., Knottenbelt, W. J., Trifunović, A., "Hypergraph partitioning for faster parallel pagerank computation", in Formal Techniques for Computer Systems and Business Processes. Springer, 2005, str. 155–171.

[27] Rungsawang, A., Manaskasemsak, B., "Fast pagerank computation on a gpu cluster", in 2012 20th Euromicro International Conference on Parallel, Distributed and Network-based Processing, 2012, str. 450-456.

[28] Morrison, J. L., Breitling, R., Higham, D. J., Gilbert, D. R., "Generank: using search engine technology for the analysis of microarray experiments", BMC bioinformatics, Vol. 6, No. 1, 2005, str. 233.

[29] Chen, Z., Cai, Z., Li, M., Liu, B., “Using search engine technology for protein function prediction”, International journal of bioinformatics research and applications, Vol. 7, No. 1, 2011, str. 101–113.

[30] Jiang, B., “Predicting human movement using pagerank in an urban environment”, 2008.